SimSun SimHei FangSong

数学建模常用算法大全

A Comprehensive Guide to Mathematical Modeling Algorithms

系统性学习教程

Nurdun Mijit

2026年6月

前言

本书目标

数学建模是将实际问题转化为数学形式，并通过数学方法求解的过程。它是连接现实世界与数学理论的桥梁，在科学研究、工程实践、经济管理等领域发挥着至关重要的作用。

本教程旨在系统性地介绍数学建模中常用的各类算法，从基础的规划优化到高级的智能启发式算法，从经典的统计分析到前沿的机器学习方法。通过本教程的学习，读者将能够：

- 理解各类算法的数学原理和工作机制
- 掌握算法的实现方法和伪代码描述
- 了解算法的适用场景和局限性
- 具备根据实际问题选择合适算法的能力
- 熟悉算法的各种变体和改进策略

读者对象

本教程适合以下读者：

- 参加数学建模竞赛的大学生和研究生
- 从事数据分析、算法研究的工程师和研究人员
- 对数学建模感兴趣的自学者
- 需要解决优化、预测、分类等问题的从业者

内容组织

本教程共分为40个部分，涵盖150余种常用算法：

第一部分 规划优化类算法：线性规划、整数规划、动态规划等经典优化方法

第二部分 智能启发式优化算法：遗传算法、粒子群、模拟退火等元启发式方法

第三部分 时序与预测类算法：回归分析、ARIMA、神经网络预测等

第四部分 聚类算法：K-Means、DBSCAN、层次聚类等无监督学习方法

第五部分 有监督分类算法：逻辑回归、SVM、随机森林等分类方法

第六部分 图论算法：最短路径、最小生成树、网络流等图算法

第七至四十部分 涵盖插值拟合、综合评价、随机仿真、微分方程、博弈论、传染病模型等更多专题

使用说明

每个算法章节都包含以下内容：

1. **数学定义**：严格的数学表述和公式推导
2. **工作原理**：算法的核心思想和执行流程
3. **伪代码**：清晰的算法步骤描述
4. **算法变体**：该算法的主要改进版本
5. **优缺点分析**：算法的优势和局限性
6. **应用领域**：典型应用场景和案例
7. **注意事项**：使用时的关键要点和常见陷阱

致谢

感谢所有为数学建模领域做出贡献的研究者和实践者。本教程参考了大量学术文献和开源资源，在此一并致谢。

Nurdun Mijit
2026年6月

目录

前言 i

I 规划优化类算法 1

1 规划优化类算法 3

1.1 线性规划 (Linear Programming, LP) . . . 3

1.1.1 数学定义 . . . 3

1.1.2 工作原理 . . . 4

1.1.3 单纯形法伪代码 . . . 4

1.1.4 对偶理论 . . . 5

1.1.5 算法变体 . . . 5

1.1.6 优缺点分析 . . . 5

1.1.7 应用领域 . . . 6

1.1.8 注意事项 . . . 6

1.2 整数规划 (Integer Programming, IP) . . . 7

1.2.1 数学定义 . . . 7

1.2.2 工作原理 . . . 7

1.2.3 分支定界法伪代码 . . . 8

1.2.4 割平面法伪代码 . . . 9

1.2.5 算法变体 . . . 9

1.2.6 优缺点分析 . . . 10

1.2.7 应用领域 . . . 10

1.2.8 注意事项 . . . 10

1.3 0-1规划 (Binary Integer Programming) . . . 11

1.3.1 数学定义 . . . 11

1.3.2 常见0-1规划问题 . . . 11

1.3.3 逻辑约束的0-1建模 . . . 12

1.3.4 优缺点与应用 . . . 12

1.4 非线性规划 (Nonlinear Programming, NLP) . . . 13

1.4.1 数学定义 . . . 13

1.4.2 最优性条件 . . . 13
1.4.3 无约束优化方法 . . . 13
1.4.4 约束优化方法 . . . 13
1.4.5 SQP算法伪代码 . . . 14
1.4.6 算法变体 . . . 14
1.4.7 优缺点分析 . . . 15
1.4.8 应用领域 . . . 15
1.4.9 注意事项 . . . 15
1.5 多目标规划 (Multi-Objective Programming, MOP) . . . 16
1.5.1 数学定义 . . . 16
1.5.2 求解方法 . . . 16
1.5.3 NSGA-II算法伪代码 . . . 17
1.5.4 优缺点分析 . . . 17
1.5.5 应用领域 . . . 18
1.6 动态规划 (Dynamic Programming, DP) . . . 18
1.6.1 数学定义 . . . 18
1.6.2 基本要素 . . . 18
1.6.3 Bellman方程 . . . 18
1.6.4 动态规划伪代码 . . . 19
1.6.5 经典应用 . . . 19
1.6.6 算法变体 . . . 20
1.6.7 优缺点分析 . . . 20
1.6.8 应用领域 . . . 20
1.6.9 注意事项 . . . 21
1.7 分枝定界法 (Branch and Bound) . . . 21
1.7.1 关键组件 . . . 21
1.8 割平面法 (Cutting Plane Method) . . . 22
1.9 本章小结 . . . 22

II 智能启发式优化算法 23

2 智能启发式优化算法 25
2.1 遗传算法 (Genetic Algorithm, GA) . . . 25
2.1.1 基本概念 . . . 25
2.1.2 工作原理 . . . 25
2.1.3 编码方式 . . . 26
2.1.4 遗传算法伪代码 . . . 27

2.1.5 选择算子 . . . 28
2.1.6 交叉算子 . . . 28
2.1.7 变异算子 . . . 28
2.1.8 算法变体 . . . 28
2.1.9 优缺点分析 . . . 29
2.1.10 应用领域 . . . 29
2.1.11 注意事项 . . . 30
2.2 粒子群优化算法 (Particle Swarm Optimization, PSO) . . . 30
2.2.1 基本概念 . . . 30
2.2.2 工作原理 . . . 30
2.2.3 PSO算法伪代码 . . . 31
2.2.4 速度更新公式详解 . . . 31
2.2.5 算法变体 . . . 32
2.2.6 优缺点分析 . . . 32
2.2.7 应用领域 . . . 33
2.2.8 注意事项 . . . 33
2.3 模拟退火算法 (Simulated Annealing, SA) . . . 33
2.3.1 基本概念 . . . 33
2.3.2 工作原理 . . . 34
2.3.3 模拟退火伪代码 . . . 34
2.3.4 关键组件 . . . 35
2.3.5 算法变体 . . . 35
2.3.6 优缺点分析 . . . 36
2.3.7 应用领域 . . . 36
2.3.8 注意事项 . . . 36
2.4 蚁群算法 (Ant Colony Optimization, ACO) . . . 37
2.4.1 基本概念 . . . 37
2.4.2 工作原理（以TSP为例） . . . 37
2.4.3 蚁群算法伪代码 . . . 38
2.4.4 算法变体 . . . 39
2.4.5 优缺点分析 . . . 39
2.4.6 应用领域 . . . 40
2.4.7 注意事项 . . . 40
2.5 禁忌搜索 (Tabu Search, TS) . . . 40
2.5.1 基本概念 . . . 40
2.5.2 工作原理 . . . 40
2.5.3 禁忌搜索伪代码 . . . 41

2.5.4 关键组件 . . . 41
2.5.5 算法变体 . . . 42
2.5.6 优缺点分析 . . . 42
2.5.7 应用领域 . . . 42
2.6 差分进化算法 (Differential Evolution, DE) . . . 43
2.6.1 基本概念 . . . 43
2.6.2 DE算法伪代码 . . . 43
2.6.3 DE策略表示 . . . 44
2.6.4 优缺点分析 . . . 44
2.7 灰狼优化算法 (Grey Wolf Optimizer, GWO) . . . 45
2.7.1 基本概念 . . . 45
2.7.2 GWO算法伪代码 . . . 45
2.7.3 优缺点 . . . 46
2.8 其他智能优化算法概述 . . . 46
2.8.1 麻雀搜索算法 (Sparrow Search Algorithm, SSA) . . . 46
2.8.2 鲸鱼优化算法 (Whale Optimization Algorithm, WOA) . . . 46
2.8.3 人工鱼群算法 (Artificial Fish Swarm Algorithm, AFSA) . . . 46
2.8.4 萤火虫算法 (Firefly Algorithm, FA) . . . 46
2.8.5 人工蜂群算法 (Artificial Bee Colony, ABC) . . . 46
2.8.6 布谷鸟搜索 (Cuckoo Search, CS) . . . 46
2.9 智能优化算法对比与选择 . . . 47
2.10 本章小结 . . . 47

III 时序与预测类算法 49

3 时序与预测类算法 51
3.1 回归分析基础 . . . 51
3.1.1 一元线性回归 . . . 51
3.1.2 多元线性回归 . . . 51
3.1.3 逐步回归算法 . . . 52
3.1.4 回归诊断 . . . 52
3.2 ARIMA模型详解 . . . 53
3.2.1 平稳性检验 . . . 53
3.2.2 模型识别 . . . 54
3.2.3 参数估计 . . . 54
3.2.4 模型诊断 . . . 54
3.2.5 预测 . . . 55

3.3 指数平滑法详解 . . . 55
3.3.1 简单指数平滑 . . . 55
3.3.2 Holt线性趋势法 . . . 55
3.3.3 Holt-Winters季节性方法 . . . 55
3.4 灰色预测 GM(1,1) 详解 . . . 56
3.4.1 适用条件 . . . 56
3.4.2 建模步骤详解 . . . 57
3.4.3 精度等级标准 . . . 58
3.4.4 改进模型 . . . 58
3.5 神经网络预测 . . . 58
3.5.1 BP神经网络架构 . . . 58
3.5.2 训练算法改进 . . . 59
3.5.3 LSTM长短期记忆网络 . . . 59
3.6 预测模型选择指南 . . . 60
3.7 模型评估指标 . . . 60

IV 聚类算法 61

4 聚类算法 63
4.1 聚类基础概念 . . . 63
4.1.1 相似性度量 . . . 63
4.1.2 聚类评估指标 . . . 64
4.2 K-Means聚类详解 . . . 64
4.2.1 算法原理 . . . 64
4.2.2 完整算法流程 . . . 65
4.2.3 K值选择方法 . . . 65
4.2.4 优缺点与改进 . . . 66
4.3 DBSCAN密度聚类详解 . . . 67
4.3.1 核心概念 . . . 67
4.3.2 参数选择 . . . 67
4.3.3 算法复杂度 . . . 67
4.3.4 优缺点分析 . . . 68
4.4 层次聚类详解 . . . 69
4.4.1 凝聚式层次聚类 . . . 69
4.4.2 链接方法（Linkage Criteria） . . . 69
4.4.3 树状图切割 . . . 70
4.5 高斯混合模型（GMM） . . . 70

4.5.1 模型定义 70
4.5.2 EM算法求解 70
4.6 谱聚类（Spectral Clustering） 71
4.6.1 算法原理 71
4.7 聚类算法选择指南 71
4.8 实际应用建议 71

V 有监督分类算法 73

5 有监督分类算法 75
5.1 Logistic逻辑回归 75
5.1.1 数学模型 75
5.1.2 参数估计 75
5.1.3 多分类扩展 75
5.2 支持向量机 (SVM) 76
5.2.1 最大间隔分类器 76
5.2.2 软间隔与核技巧 76
5.3 决策树 76
5.3.1 ID3/C4.5算法 76
5.3.2 CART算法 77
5.3.3 剪枝策略 77
5.4 随机森林 77
5.4.1 算法原理 77
5.4.2 优缺点 77
5.5 K近邻 (KNN) 77
5.5.1 算法描述 77
5.5.2 优缺点 78
5.6 朴素贝叶斯 78
5.6.1 贝叶斯定理 78
5.6.2 条件独立性假设 78
5.6.3 变体 78
5.7 AdaBoost提升算法 78
5.7.1 算法原理 78
5.8 分类模型评估 79

VI 图论算法 81

6 图论算法 83

6.1 最短路径算法 . . . 83
6.1.1 Dijkstra算法 . . . 83
6.1.2 Bellman-Ford算法 . . . 84
6.1.3 Floyd-Warshall算法 . . . 84
6.1.4 SPFA算法 . . . 85
6.2 最小生成树 . . . 85
6.2.1 Kruskal算法 . . . 85
6.2.2 Prim算法 . . . 86
6.3 网络流算法 . . . 87
6.3.1 最大流问题 . . . 87
6.3.2 Ford-Fulkerson方法 . . . 87
6.3.3 Dinic算法 . . . 88
6.3.4 最小费用最大流 . . . 88
6.4 二分图匹配 . . . 88
6.4.1 匈牙利算法 . . . 88
6.4.2 Kuhn-Munkres算法 (KM算法) . . . 89
6.5 关键路径法 (CPM) . . . 89
6.6 图论算法应用 . . . 90

VII 插值与曲线拟合 91

7 插值与曲线拟合 93

7.1 拉格朗日插值 . . . 93
7.1.1 基本原理 . . . 93
7.1.2 性质 . . . 93
7.1.3 优缺点 . . . 93
7.2 牛顿插值 . . . 94
7.2.1 差商定义 . . . 94
7.2.2 牛顿插值多项式 . . . 94
7.2.3 算法优势 . . . 94
7.3 分段线性插值 . . . 95
7.3.1 基本原理 . . . 95
7.4 三次样条插值 . . . 95
7.4.1 定义 . . . 95

7.4.2 边界条件 . . . 95
7.4.3 三弯矩方程 . . . 95
7.4.4 优势 . . . 96
7.5 埃尔米特插值 . . . 96
7.5.1 问题描述 . . . 96
7.5.2 构造方法 . . . 96
7.6 最小二乘拟合 . . . 96
7.6.1 基本原理 . . . 96
7.6.2 线性最小二乘 . . . 97
7.6.3 非线性最小二乘 . . . 97
7.6.4 常用拟合模型 . . . 98
7.7 方法选择指南 . . . 98
7.8 注意事项 . . . 98

VIII 综合评价与多准则决策 99

8 综合评价与多准则决策 101
8.1 AHP层次分析法详解 . . . 101
8.1.1 基本原理 . . . 101
8.1.2 判断矩阵构建 . . . 101
8.1.3 权重计算 . . . 102
8.1.4 随机一致性指标 RI . . . 103
8.1.5 层次总排序 . . . 103
8.1.6 优缺点分析 . . . 104
8.2 熵权法 (Entropy Weight Method) . . . 104
8.2.1 基本原理 . . . 104
8.2.2 计算步骤 . . . 105
8.2.3 注意事项 . . . 105
8.3 TOPSIS法详解 . . . 106
8.3.1 基本原理 . . . 106
8.3.2 计算步骤 . . . 106
8.3.3 示例 . . . 106
8.4 灰色关联分析 (GRA) . . . 107
8.4.1 基本原理 . . . 107
8.4.2 计算步骤 . . . 107
8.5 模糊综合评价 (FCE) . . . 108
8.5.1 基本概念 . . . 108

8.5.2 实施步骤 . . . 108
8.5.3 隶属函数设计 . . . 108
8.6 DEA数据包络分析 . . . 109
8.6.1 基本原理 . . . 109
8.6.2 CCR模型（规模报酬不变） . . . 109
8.6.3 效率解读 . . . 109
8.7 VIKOR方法 . . . 109
8.7.1 基本原理 . . . 109
8.7.2 计算步骤 . . . 110
8.8 方法选择指南 . . . 110
8.9 组合评价方法 . . . 110

IX 随机仿真与概率模型 111

9 随机仿真与概率模型 113
9.1 蒙特卡洛方法 (Monte Carlo Method) . . . 113
9.1.1 基本原理 . . . 113
9.1.2 数值积分示例 . . . 113
9.1.3 圆周率 π 估算 . . . 114
9.1.4 方差缩减技术 . . . 114
9.2 马尔可夫链 (Markov Chain) . . . 114
9.2.1 基本定义 . . . 114
9.2.2 转移矩阵 . . . 115
9.2.3 稳态分布 . . . 115
9.2.4 应用实例 . . . 115
9.3 排队论 (Queueing Theory) . . . 116
9.3.1 Kendall记号 . . . 116
9.3.2 M/M/1模型详解 . . . 116
9.3.3 M/M/c模型 . . . 117
9.3.4 M/G/1模型 . . . 117
9.3.5 排队网络 . . . 117
9.4 随机过程仿真 . . . 118
9.4.1 离散事件仿真 . . . 118
9.4.2 随机数生成 . . . 118
9.5 风险评估应用 . . . 119
9.5.1 VaR (Value at Risk) 计算 . . . 119
9.5.2 可靠性分析 . . . 119

9.6 实际应用建议 119

X 微分方程数值解法 121

XI 数据降维算法 123

XII 其他拓展建模算法 125

XIII 复杂系统演化类 127

XIV 博弈论建模算法 129

XV 传染病与种群动力学 131

XVI 离散事件系统仿真 133

10 离散事件系统仿真 135

10.1 事件步长法 135
10.2 固定步长法 135
10.3 Petri网 135

11 可靠性与故障建模 137

11.1 故障树分析 (FTA) 137
11.2 威布尔分布 137
11.3 蒙特卡洛可靠性仿真 137

12 库存与供应链专用建模 139

12.1 EOQ经济订货批量 139
12.2 报童模型 139
12.3 (s,S)策略 139

13 数理统计检验与抽样 141

13.1 假设检验 141
13.2 Bootstrap自助法 141

14 流体、交通专用建模 143

14.1 LWR交通流模型 143
14.2 IDM智能驾驶员模型 143

XVII 补充智能优化算法 145

15 补充智能优化算法 147
15.1 人工蜂群算法 (ABC) 147
15.2 布谷鸟搜索 (CS) 147
15.3 教与学优化 (TLBO) 147

16 模糊拓展与粗糙集衍生 149
16.1 模糊层次分析法 (FAHP) 149
16.2 变精度粗糙集 (VPRS) 149

17 信号与数据处理 151
17.1 快速傅里叶变换 (FFT) 151
17.2 小波阈值降噪 151

18 组合优化与组合数学 153
18.1 容斥原理 153
18.2 回溯算法 153

19 图像处理类建模 155
19.1 Otsu大津法 155
19.2 形态学运算 155

20 偏微分方程补充数值解法 157
20.1 有限体积法 (FVM) 157
20.2 谱方法 157

21 计量经济学建模 159
21.1 ADF单位根检验 159
21.2 协整检验 159
21.3 GARCH模型 159

22 最优控制与变分类 161
22.1 变分法 161
22.2 庞特里亚金极小值原理 161
22.3 线性二次调节器 (LQR) 161

23 鲁棒优化与不确定规划 **163**
23.1 鲁棒线性规划 163
23.2 机会约束规划 163

24 高级多目标优化算法 **165**
24.1 NSGA-II详解 165
24.2 MOPSO多目标粒子群 165

XVIII 空间地理与GIS建模 **167**

25 空间地理与GIS建模 **169**
25.1 克里金插值 (Kriging) 169
25.2 反距离加权 (IDW) 169
25.3 泰森多边形 (Voronoi Diagram) 169
25.4 空间自相关 169
25.5 CA-Markov耦合模型 170

26 高级排队与服务系统 **171**
26.1 M/M/1排队模型 171
26.2 M/M/s模型 171
26.3 Jackson排队网络 171

27 生物医学建模专项 **173**
27.1 药代动力学房室模型 173
27.2 生存分析 173

28 稳健统计与异常值处理 **175**
28.1 M估计 175
28.2 Grubbs检验 175

29 滤波算法 **177**
29.1 卡尔曼滤波 (KF) 177
29.2 扩展卡尔曼滤波 (EKF) 177
29.3 粒子滤波 (PF) 177

30 组合博弈与合作博弈 **179**
30.1 Shapley值 179

31 水文与环境建模 **181**
31.1 马斯京根河道演算 181

31.2 SCS径流曲线法 181

32 稀疏优化与压缩感知 183
32.1 Lasso (L1正则化) 183
32.2 Ridge回归 (L2正则化) 183
32.3 弹性网 (Elastic Net) 183

33 马尔可夫衍生拓展模型 185
33.1 隐马尔可夫模型 (HMM) 185
33.2 吸收马尔可夫链 185

34 小众元启发式优化补充 187
34.1 水循环算法 (WCA) 187
34.2 热交换优化 (TEO) 187
34.3 原子搜索优化 (ASO) 187

A 数学基础 189
A.1 线性代数 189
A.2 概率论 189
A.3 微积分 189
A.4 优化理论 189

B 常用软件工具 191
B.1 MATLAB 191
B.2 Python 191
B.3 R语言 191
B.4 LINGO/GUROBI 191

C 数学建模案例分析 193
C.1 案例一：城市交通拥堵评估 193
C.1.1 问题描述 193
C.1.2 建模思路 193
C.1.3 模型实现 194
C.1.4 改善建议 194
C.2 案例二：传染病预测与控制 194
C.2.1 问题背景 194
C.2.2 SEIR模型建立 194
C.2.3 参数估计 195
C.2.4 情景分析 195

C.2.5 疫苗接种策略 195
C.3 案例三：供应链优化 195
C.3.1 问题描述 195
C.3.2 混合整数规划模型 195
C.3.3 求解结果 196
C.4 案例四：股票价格预测 196
C.4.1 数据预处理 196
C.4.2 LSTM模型构建 196
C.4.3 评估结果 197

D 算法实现技巧与常见陷阱 199
D.1 数值计算注意事项 199
D.1.1 精度问题 199
D.1.2 收敛性问题 199
D.2 智能优化算法调参指南 199
D.2.1 遗传算法 199
D.2.2 粒子群优化 199
D.2.3 模拟退火 200
D.3 机器学习建模最佳实践 200
D.3.1 数据划分 200
D.3.2 过拟合预防 200
D.3.3 特征工程 200
D.4 常见错误与避免方法 201
D.4.1 建模常见错误 201
D.5 论文写作建议 201
D.5.1 模型描述要点 201
D.5.2 结果展示技巧 202
D.5.3 灵敏度分析 202
D.6 常用软件工具对比 202
D.7 数学建模竞赛策略 202
D.7.1 时间分配建议（3天赛制） 202
D.7.2 团队协作 203
D.7.3 摘要写作要点 203

Part I

规划优化类算法

Chapter 1

规划优化类算法

规划优化是数学建模中最核心的内容之一，旨在在给定约束条件下寻找目标函数的最优解。本章介绍线性规划、整数规划、非线性规划、动态规划等经典优化方法。

1.1 线性规划 (Linear Programming, LP)

1.1.1 数学定义

定义 1.1 (线性规划标准型)**.** 线性规划问题的标准形式为：

$$\begin{aligned} \min \quad & z = \mathbf{c}^T\mathbf{x} = \sum_{j=1}^{n} c_j x_j \\ s.t. \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned} \tag{1.1}$$

其中：

- $\mathbf{x} = (x_1, x_2, \ldots, x_n)^T$ 为决策变量向量
- $\mathbf{c} = (c_1, c_2, \ldots, c_n)^T$ 为目标函数系数向量
- A 为 $m \times n$ 的约束矩阵
- $\mathbf{b} = (b_1, b_2, \ldots, b_m)^T$ 为资源向量，且 $\mathbf{b} \geq \mathbf{0}$

注释 1.1. 任何线性规划问题都可以通过以下变换转化为标准型：

1. 最大化问题：将目标函数取负号转化为最小化
2. 不等式约束：引入松弛变量或剩余变量转化为等式
3. 无约束变量：用两个非负变量之差表示
4. 负右端项：约束两边同乘-1

1.1.2 工作原理

线性规划的可行域是一个凸多面体（凸集），最优解一定在可行域的顶点（基本可行解）处取得。单纯形法通过沿着可行域的边从一个顶点移动到相邻的更优顶点，直到找到最优解。

核心概念：

- **基本解：**令$n-m$个变量为0，求解m个方程得到的解
- **基本可行解：**满足非负约束的基本解，对应可行域的顶点
- **最优性条件：**所有检验数（reduced cost）非负时达到最优

1.1.3 单纯形法伪代码

Algorithm 1.1 单纯形法 (Simplex Method)

输入: 标准型线性规划问题：$\min \mathbf{c}^T\mathbf{x}$, s.t. $A\mathbf{x}=\mathbf{b}, \mathbf{x}\geq \mathbf{0}$

输出: 最优解 $\mathbf{x}^*$ 和最优值 z^*

初始化: 找到一个初始基本可行解，确定基变量集合 B 和非基变量集合 N

计算检验数：$\mathbf{r}_N = \mathbf{c}_N - \mathbf{c}_B^T B^{-1} N$
while 存在 $r_j < 0$ （对于某个非基变量 $j \in N$） **do**
    选择进基变量：$j^* = \arg\min_{j\in N} r_j$ （最负检验数规则）
    计算方向：$\mathbf{d} = B^{-1}\mathbf{a}_{j^*}$
    if $\mathbf{d} \leq \mathbf{0}$ **then**
        return “问题无界”
    end if
    确定离基变量（最小比值测试）：
    $i^* = \arg\min_{i:d_i>0} \frac{(B^{-1}\mathbf{b})_i}{d_i}$
    更新基变量集合：将 j^* 加入 B，将 i^* 移出 B
    更新 B^{-1} 和基本解
    重新计算检验数 $\mathbf{r}_N$
end while
return 当前基本可行解 $\mathbf{x}^*$ 和目标值 $z^* = \mathbf{c}^T\mathbf{x}^*$

1.1.4 对偶理论

定义 1.2 (对偶问题). 原问题（*Primal*）：

$$\begin{aligned} \min \quad & \mathbf{c}^T\mathbf{x} \\ s.t. \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned} \tag{1.2}$$

对偶问题（*Dual*）：

$$\begin{aligned} \max \quad & \mathbf{b}^T\mathbf{y} \\ s.t. \quad & A^T\mathbf{y} \leq \mathbf{c} \end{aligned} \tag{1.3}$$

定理 1.1 (强对偶定理). 如果原问题有最优解 $\mathbf{x}^*$，则对偶问题也有最优解 $\mathbf{y}^*$，且两者目标函数值相等：

$$\mathbf{c}^T\mathbf{x}^* = \mathbf{b}^T\mathbf{y}^* \tag{1.4}$$

1.1.5 算法变体

1. **两阶段法**：当难以找到初始基本可行解时，第一阶段构造人工问题寻找可行解，第二阶段求解原问题。

2. **大M法**：通过引入人工变量和惩罚项M，将寻找初始可行解融入单一阶段。

3. **对偶单纯形法**：从对偶可行解出发，保持对偶可行性，逐步达到原始可行性。适用于添加新约束后的重新优化。

4. **内点法**：从可行域内部逼近最优解，包括：

   - 路径跟踪法（Path-following method）
   - 势垒法（Barrier method）
   - Karmarkar算法

   内点法对大规模问题具有多项式时间复杂度优势。

5. **修订单纯形法**：不直接存储整个表格，而是维护基逆矩阵 B^{-1}，节省存储空间。

1.1.6 优缺点分析

优点：

- 理论基础完善，有严格的收敛性保证

- 对于中小规模问题，单纯形法通常非常高效
- 可以提供灵敏度分析信息（影子价格、允许变化范围）
- 商业求解器（如CPLEX、Gurobi）高度优化，可处理百万级变量

缺点：

- 单纯形法在最坏情况下是指数时间复杂度
- 对于超大规模问题，内存需求可能过大
- 退化现象可能导致循环或收敛缓慢
- 仅适用于线性关系，无法直接处理非线性问题

1.1.7 应用领域

- **生产计划**：产品组合优化、资源分配
- **运输问题**：最小成本运输方案、物流网络设计
- **饮食问题**：营养配比优化、饲料配方
- **金融投资**：投资组合优化（Markowitz模型的线性近似）
- **能源调度**：电力机组组合、炼油厂优化
- **广告预算分配**：媒体投放优化

1.1.8 注意事项

1. **模型构建**：确保所有关系确实是线性的，必要时进行线性化近似
2. **数值稳定性**：避免系数数量级差异过大，适当缩放变量
3. **退化处理**：使用Bland规则或其他抗退化策略避免循环
4. **灵敏度分析**：利用对偶变量（影子价格）分析资源价值
5. **不可行诊断**：当问题不可行时，使用Farkas引理或弹性约束定位冲突
6. **软件选择**：小规模可用Excel Solver，中大规模推荐使用专业求解器

1.2 整数规划 (Integer Programming, IP)

1.2.1 数学定义

定义 1.3 (整数规划). 整数规划是指部分或全部决策变量被限制为整数值的线性规划问题：

$$\begin{aligned} \min \quad & \mathbf{c}^T\mathbf{x} \\ s.t. \quad & A\mathbf{x} \leq \mathbf{b} \\ & x_j \in \mathbb{Z}, \quad \forall j \in I \subseteq \{1, 2, \ldots, n\} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned} \tag{1.5}$$

分类：

- **纯整数规划 *(Pure IP)***：$I = \{1, 2, \ldots, n\}$，所有变量均为整数
- **混合整数规划 *(MIP/MILP)***：$I \subset \{1, 2, \ldots, n\}$，部分变量为整数
- ***0-1*规划 *(Binary IP)***：$x_j \in \{0, 1\}$，变量仅取*0*或*1*

1.2.2 工作原理

整数规划的可行域是离散点集，不再是凸集，因此不能直接使用单纯形法。主要求解策略包括：

1. 分支定界法 (Branch and Bound)

核心思想是通过系统性地枚举候选解空间来寻找最优解：

- **分支**：将问题分解为子问题（通常通过对某个分数变量添加整数约束）
- **定界**：通过求解线性松弛问题获得上下界
- **剪枝**：当子问题的界限表明其不可能包含更优解时，舍弃该子树

2. 割平面法 (Cutting Plane)

通过添加额外的线性约束（割平面）来切割掉非整数解区域，同时保留所有整数可行解，逐步逼近整数最优解。

1.2.3 分支定界法伪代码

Algorithm 1.2 分支定界法 (Branch and Bound)

输入: 整数规划问题

输出: 最优整数解 $\mathbf{x}^*$ 和最优值 z^*

初始化: 初始化全局上界 $UB = +\infty$，下界 $LB = -\infty$

创建根节点，对应原问题的线性松弛
将根节点加入待处理节点列表 L
while L 非空 **do**
    从 L 中选择一个节点 P （节点选择策略）
    从 L 中移除 P
    求解节点 P 的线性松弛问题，得到解 $\mathbf{x}_P$ 和目标值 z_P
    if 松弛问题不可行 **then**
        剪枝：丢弃节点 P

    end if
    if $z_P \geq UB$ **then**
        剪枝：该节点不可能产生更优解

    end if
    if $\mathbf{x}_P$ 满足所有整数约束 **then**
        找到新的整数可行解
        更新 $UB = z_P$，$\mathbf{x}^* = \mathbf{x}_P$
        剪枝：丢弃节点 P

    end if
    分支：选择某个分数变量 x_k （分支变量选择策略）
    创建两个子节点：
      左子节点：添加约束 $x_k \leq \lfloor x_{P,k} \rfloor$
      右子节点：添加约束 $x_k \geq \lceil x_{P,k} \rceil$
    将两个子节点加入 L
end while
if $UB < +\infty$ **then**
    return $\mathbf{x}^*$, $z^* = UB$
else
    return “问题不可行”
end if

1.2.4 割平面法伪代码

Algorithm 1.3 Gomory割平面法

输入: 整数规划问题

输出: 最优整数解 $\mathbf{x}^*$

求解线性松弛问题，得到最优单纯形表
while 当前解 $\mathbf{x}$ 不满足整数约束 **do**
    选择一个分数基变量 x_i （x_i 不是整数）
    从单纯形表中提取第 i 行：
    $x_i + \sum_{j \in N} a_{ij} x_j = b_i$
    生成Gomory割平面：
    $\sum_{j \in N} (a_{ij} - \lfloor a_{ij} \rfloor) x_j \geq b_i - \lfloor b_i \rfloor$
    将割平面添加到问题中（引入松弛变量）
    用对偶单纯形法重新求解
    if 问题变为不可行 **then**
        return “原问题无整数可行解”
    end if
end while
return 当前解 $\mathbf{x}^* = \mathbf{x}$

1.2.5 算法变体

1. **分支切割法 (Branch and Cut)**：结合分支定界和割平面法，在每个节点添加割平面收紧松弛。

2. **分支定价法 (Branch and Price)**：结合分支定界和列生成，适用于变量极多的问题。

3. **启发式方法**：

    - 舍入启发式：将线性松弛解舍入到最近整数
    - 局部搜索：在整数解邻域内搜索改进
    - 禁忌搜索、遗传算法等元启发式

4. **动态规划**：对于特定结构的整数规划（如背包问题），可用动态规划精确求解。

5. **拉格朗日松弛**：放松某些复杂约束，将其加入目标函数作为惩罚项。

1.2.6 优缺点分析

优点：

- 能够精确建模需要整数决策的实际问题
- 分支定界法可以保证找到全局最优解
- 现代MIP求解器性能强大，可处理大规模问题
- 可以灵活表达逻辑约束（通过0-1变量）

缺点：

- NP-hard问题，最坏情况下求解时间是指数级的
- 对于大规模或结构复杂的问题，可能需要很长时间才能找到最优解
- 分支定界树的规模可能非常大，消耗大量内存
- 线性松弛的质量直接影响求解效率

1.2.7 应用领域

- **设施选址**：决定在哪些位置建立工厂、仓库
- **任务分配**：人员-任务匹配、机器调度
- **背包问题**：资源选择、投资组合
- **旅行商问题(TSP)**：路径优化、物流配送
- **生产调度**：作业车间调度、流水车间调度
- **网络设计**：通信网络、交通网络规划
- **资本预算**：项目选择、投资决策

1.2.8 注意事项

1. **模型 formulation**：好的建模方式可以显著改善线性松弛质量
2. **预处理**：利用约束传播、系数约简等技术简化问题
3. **变量选择**：分支变量的选择策略影响搜索效率
4. **节点选择**：最佳优先、深度优先、广度优先各有优劣

5. **对称性处理**：对称结构会导致大量等价解，需添加对称性破缺约束

6. **大M技巧**：使用足够小但安全的M值，避免数值问题

7. **求解器参数**：调整gap容忍度、时间限制、启发式频率等参数

1.3 0-1规划 (Binary Integer Programming)

1.3.1 数学定义

定义 1.4 (0-1规划). *0-1规划是整数规划的特例，所有决策变量只能取值0或1*:

$$\begin{aligned} \min \quad & \mathbf{c}^T\mathbf{x} \\ s.t. \quad & A\mathbf{x} \leq \mathbf{b} \\ & x_j \in \{0,1\}, \quad j = 1,2,\ldots,n \end{aligned} \tag{1.6}$$

0-1变量常用于表示“是/否”决策：$x_j = 1$ 表示选择第 j 个选项，$x_j = 0$ 表示不选择。

1.3.2 常见0-1规划问题

1. 背包问题 (Knapsack Problem)

$$\begin{aligned} \max \quad & \sum_{j=1}^{n} v_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^{n} w_j x_j \leq W \\ & x_j \in \{0,1\} \end{aligned} \tag{1.7}$$

其中 v_j 为物品价值，w_j 为物品重量，W 为背包容量。

2. 指派问题 (Assignment Problem)

$$\begin{aligned} \min \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j=1}^{n} x_{ij} = 1, \quad i = 1,\ldots,n \\ & \sum_{i=1}^{n} x_{ij} = 1, \quad j = 1,\ldots,n \\ & x_{ij} \in \{0,1\} \end{aligned} \tag{1.8}$$

3. 集合覆盖问题 (Set Covering)

$$\begin{aligned} \min \quad & \sum_{j=1}^{n} c_j x_j \\ \text{s.t.} \quad & \sum_{j:i\in S_j} x_j \geq 1, \quad \forall i \in U \\ & x_j \in \{0,1\} \end{aligned} \tag{1.9}$$

1.3.3 逻辑约束的0-1建模

0-1变量可以表达各种逻辑关系：

表 1.1: 常见逻辑约束的0-1建模

逻辑关系	数学表达
如果 $x_1 = 1$ 则 $x_2 = 1$	$x_1 \leq x_2$
x_1 和 x_2 不能同时为1	$x_1 + x_2 \leq 1$
至少选择一个	$\sum x_j \geq 1$
恰好选择k个	$\sum x_j = k$
如果 $x_1 = 1$ 则 $y \leq b$	$y \leq b + M(1 - x_1)$
$x = 1$ 当且仅当 $y > 0$	$y \leq Mx, y \geq \epsilon x$

1.3.4 优缺点与应用

优点：

- 简洁地表达二元决策
- 可以建模复杂的逻辑约束
- 线性松弛提供了较好的边界

缺点：

- 变量数量可能非常多
- 某些问题的线性松弛很弱
- NP-hard，大规模问题难以精确求解

应用： 项目选择、开关决策、路径选择、设施开启/关闭等。

1.4 非线性规划 (Nonlinear Programming, NLP)

1.4.1 数学定义

定义 1.5 (非线性规划). 非线性规划是指目标函数或约束条件中至少有一个是非线性函数的优化问题：

$$\begin{aligned} \min \quad & f(\mathbf{x}) \\ s.t. \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \ldots, m \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \ldots, p \\ & \mathbf{x} \in \mathbb{R}^n \end{aligned} \tag{1.10}$$

其中 f, g_i, h_j 中至少有一个是非线性函数。

1.4.2 最优性条件

定理 1.2 (KKT条件 (Karush-Kuhn-Tucker)). 对于满足某些正则性条件的非线性规划问题，局部最优解 $\mathbf{x}^*$ 必须满足：

1. **平稳性：** $\nabla f(\mathbf{x}^*) + \sum_{i=1}^{m} \lambda_i \nabla g_i(\mathbf{x}^*) + \sum_{j=1}^{p} \mu_j \nabla h_j(\mathbf{x}^*) = \mathbf{0}$
2. **原始可行性：** $g_i(\mathbf{x}^*) \leq 0, h_j(\mathbf{x}^*) = 0$
3. **对偶可行性：** $\lambda_i \geq 0$
4. **互补松弛：** $\lambda_i g_i(\mathbf{x}^*) = 0$

1.4.3 无约束优化方法

1. 梯度下降法

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k) \tag{1.11}$$

2. 牛顿法

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k) \tag{1.12}$$

3. 拟牛顿法 (BFGS) 用近似矩阵 B_k 替代Hessian矩阵的逆：

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k B_k \nabla f(\mathbf{x}_k) \tag{1.13}$$

1.4.4 约束优化方法

1. **罚函数法：** 将约束违反量加入目标函数

$$P(\mathbf{x}) = f(\mathbf{x}) + \rho \sum_i [\max(0, g_i(\mathbf{x}))]^2 \tag{1.14}$$

2. **障碍函数法：**在可行域边界设置障碍

$$B(\mathbf{x}) = f(\mathbf{x}) - \mu \sum_i \log(-g_i(\mathbf{x})) \tag{1.15}$$

3. **增广拉格朗日法：**结合拉格朗日乘子和罚函数

4. **序列二次规划 (SQP)：**每步求解一个二次规划子问题

1.4.5 SQP算法伪代码

Algorithm 1.4 序列二次规划 (SQP)

输入：初始点 $\mathbf{x}_0$，初始乘子 $\boldsymbol{\lambda}_0, \boldsymbol{\mu}_0$

输出：局部最优解 $\mathbf{x}^*$

$k \leftarrow 0$

while 未收敛 **do**

　计算梯度 $\nabla f(\mathbf{x}_k)$ 和约束梯度

　构建二次规划子问题：

$$\begin{aligned} \min_{\mathbf{d}} \quad & \nabla f(\mathbf{x}_k)^T \mathbf{d} + \frac{1}{2} \mathbf{d}^T H_k \mathbf{d} \\ \text{s.t.} \quad & g_i(\mathbf{x}_k) + \nabla g_i(\mathbf{x}_k)^T \mathbf{d} \le 0 \\ & h_j(\mathbf{x}_k) + \nabla h_j(\mathbf{x}_k)^T \mathbf{d} = 0 \end{aligned}$$

　求解QP子问题，得到搜索方向 $\mathbf{d}_k$

　线搜索确定步长 α_k

　更新 $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$

　更新乘子 $\boldsymbol{\lambda}_{k+1}, \boldsymbol{\mu}_{k+1}$

　更新Hessian近似 H_k （BFGS更新）

　$k \leftarrow k + 1$

end while

return $\mathbf{x}^* = \mathbf{x}_k$

1.4.6 算法变体

- **内点法：**用于凸非线性规划，多项式时间复杂度
- **信赖域法：**限制步长在信赖域内，保证模型有效性
- **共轭梯度法：**适合大规模无约束优化

- **坐标下降法**：每次优化一个变量，简单但收敛慢
- **随机梯度下降 (SGD)**：用于大规模机器学习优化

1.4.7 优缺点分析

优点：

- 能够处理更一般的非线性关系
- 更符合实际问题的物理规律
- 有丰富的理论和算法支持

缺点：

- 可能存在多个局部最优解
- 收敛性和全局最优性难以保证
- 计算复杂度高于线性规划
- 对初始点敏感
- 需要计算梯度和Hessian矩阵

1.4.8 应用领域

- **工程设计**：结构优化、参数整定
- **化工过程**：反应器优化、分离过程
- **金融**：期权定价、风险管理
- **机器学习**：神经网络训练、支持向量机
- **轨迹优化**：机器人路径、航天器轨道
- **经济均衡**：一般均衡模型

1.4.9 注意事项

1. **凸性检查**：凸问题保证全局最优，尽量构造凸模型
2. **初始点选择**：多起点运行以避免局部最优
3. **尺度问题**：变量和目标函数量级应相当

4. **导数计算**：使用自动微分提高精度和效率

5. **约束处理**：合理选择罚参数或障碍参数

6. **收敛判据**：同时检查梯度范数和步长大小

1.5 多目标规划 (Multi-Objective Programming, MOP)

1.5.1 数学定义

定义 1.6 (多目标优化). 多目标优化问题包含多个相互冲突的目标函数：

$$\begin{aligned} \min \quad & \mathbf{F}(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \ldots, f_k(\mathbf{x})]^T \\ s.t. \quad & \mathbf{x} \in X \end{aligned} \tag{1.16}$$

定义 1.7 (Pareto最优). 解 $\mathbf{x}^*$ 称为*Pareto*最优（非支配解），如果不存在另一个可行解 $\mathbf{x}$ 使得：

- $f_i(\mathbf{x}) \leq f_i(\mathbf{x}^*)$ 对所有 i 成立
- 且至少存在一个 j 使得 $f_j(\mathbf{x}) < f_j(\mathbf{x}^*)$

1.5.2 求解方法

1. 加权和方法

$$\min \sum_{i=1}^{k} w_i f_i(\mathbf{x}), \quad w_i \geq 0, \sum w_i = 1 \tag{1.17}$$

通过改变权重向量获得不同的Pareto解。

2. ϵ-约束法

$$\begin{aligned} \min \quad & f_1(\mathbf{x}) \\ \text{s.t.} \quad & f_i(\mathbf{x}) \leq \epsilon_i, \quad i = 2, \ldots, k \\ & \mathbf{x} \in X \end{aligned} \tag{1.18}$$

3. 目标规划法 设定每个目标的目标值 T_i，最小化偏差：

$$\min \sum_{i=1}^{k} w_i |f_i(\mathbf{x}) - T_i| \tag{1.19}$$

4. Pareto前沿搜索 使用进化算法直接搜索Pareto最优解集。

1.5.3 NSGA-II算法伪代码

Algorithm 1.5 NSGA-II 多目标遗传算法

输入: 种群大小 N，最大代数 G_{max}

输出: Pareto最优解集

初始化: 随机生成初始种群 P_0，大小为 N

for $gen = 1$ to G_{max} **do**
  对 P_{gen-1} 进行选择、交叉、变异，生成子代 Q_{gen}
  合并种群：$R_{gen} = P_{gen-1} \cup Q_{gen}$
  对 R_{gen} 进行快速非支配排序，得到前沿 $F_1, F_2, \ldots$
  初始化新种群 $P_{gen} = \emptyset$
  for 每个前沿 F_i （按顺序） **do**
    if $|P_{gen}| + |F_i| \leq N$ **then**
      $P_{gen} \leftarrow P_{gen} \cup F_i$
    else
      计算 F_i 中个体的拥挤距离
      按拥挤距离降序排列 F_i
      选择前 $N - |P_{gen}|$ 个个体加入 P_{gen}

    end if
  end for
end for
return F_1 （第一前沿，即Pareto最优解集）

1.5.4 优缺点分析

优点：

- 能够处理多个冲突目标
- 提供Pareto解集供决策者选择
- 揭示目标之间的权衡关系

缺点：

- 没有唯一最优解，需要额外决策
- Pareto前沿可能难以完全获得
- 高维目标空间可视化困难
- 计算成本高

1.5.5 应用领域

产品设计（成本vs质量vs性能）、投资组合（收益vs风险）、环境管理（经济vs生态）、工程设计（强度vs重量vs成本）等。

1.6 动态规划 (Dynamic Programming, DP)

1.6.1 数学定义

定义 1.8 (动态规划). *动态规划是将多阶段决策问题分解为一系列子问题，利用最优子结构和重叠子问题性质，通过递推关系求解的方法。*

*Bellman*最优性原理：一个最优策略的子策略也必须是最优的。

1.6.2 基本要素

1. **阶段**：将问题划分为若干个决策阶段 $t = 1, 2, \ldots, T$

2. **状态**：描述系统在每阶段的状况 s_t

3. **决策**：每阶段的选择 x_t

4. **状态转移**：$s_{t+1} = T_t(s_t, x_t)$

5. **阶段收益**：$r_t(s_t, x_t)$

6. **最优值函数**：$V_t(s_t)$ 表示从状态 s_t 开始到结束的最大总收益

1.6.3 Bellman方程

$$V_t(s_t) = \max_{x_t}\{r_t(s_t, x_t) + V_{t+1}(s_{t+1})\} \tag{1.20}$$

边界条件：$V_T(s_T) = \max_{x_T} r_T(s_T, x_T)$

1.6.4 动态规划伪代码

Algorithm 1.6 动态规划（逆向递推）

输入: 阶段数 T，状态空间 S，决策空间 X

输出: 最优策略和最优值

初始化： 计算终端值 $V_T(s_T)$ 对所有 $s_T \in S$
逆向递推：
for $t = T-1$ down to 1 **do**
  for 每个状态 $s_t \in S$ **do**
    $V_t(s_t) \leftarrow -\infty$
    for 每个可行决策 $x_t \in X(s_t)$ **do**
      计算下一状态：$s_{t+1} = T_t(s_t, x_t)$
      计算总收益：$value = r_t(s_t, x_t) + V_{t+1}(s_{t+1})$
      if $value > V_t(s_t)$ **then**
        $V_t(s_t) \leftarrow value$
        记录最优决策：$x_t^*(s_t) \leftarrow x_t$
      end if
    end for
  end for
end for
正向追踪：
从初始状态 s_1 开始
for $t = 1$ to T **do**
  执行最优决策 $x_t^* = x_t^*(s_t)$
  更新状态：$s_{t+1} = T_t(s_t, x_t^*)$
end for
return 最优策略 $\{x_1^*, \ldots, x_T^*\}$ 和最优值 $V_1(s_1)$

1.6.5 经典应用

1. 最短路径问题

$$V(i) = \min_{j \in \text{succ}(i)} \{c_{ij} + V(j)\} \tag{1.21}$$

2. 背包问题

$$V(k, w) = \max\{V(k-1, w), v_k + V(k-1, w-w_k)\} \tag{1.22}$$

3. 资源分配

$$V(k,r) = \max_{0 \le x \le r} \{f_k(x) + V(k-1, r-x)\} \tag{1.23}$$

1.6.6 算法变体

- **记忆化搜索**：自顶向下，递归+缓存
- **表格法**：自底向上，填表
- **值迭代**：无限 horizon 问题
- **策略迭代**：交替评估和改进策略
- **近似动态规划**：用函数近似处理大规模状态空间
- **强化学习**：模型未知时的动态规划

1.6.7 优缺点分析

优点：

- 保证找到全局最优解
- 能够处理复杂的时序决策
- 提供完整的策略映射

缺点：

- **维数灾难**：状态空间随维度指数增长
- 需要知道完整的系统模型
- 离散化会引入误差
- 存储需求大

1.6.8 应用领域

库存控制、设备更换、投资组合、路径规划、生物序列比对、语音识别、游戏AI等。

1.6.9 注意事项

1. **状态定义**：状态应包含做出最优决策所需的全部信息
2. **状态空间离散化**：平衡精度和计算量
3. **边界条件**：正确设置终端条件
4. **存储优化**：有时只需保存相邻层的值
5. **并行化**：同一层的状态计算可以并行

1.7 分枝定界法 (Branch and Bound)

详见整数规划节。此处补充更多细节。

1.7.1 关键组件

1. 界限计算

- 上界（最小化问题）：任意可行解的目标值
- 下界：线性松弛的最优值

2. 分支策略

- 最分数变量分支
- 伪成本分支
- 强分支

3. 节点选择策略

- 最佳优先（Best-first）：选择界限最好的节点
- 深度优先（Depth-first）：减少内存使用
- 广度优先（Breadth-first）：快速提升全局下界

1.8 割平面法 (Cutting Plane Method)

详见整数规划节。常用割平面类型：

1. **Gomory割**：从单纯形表生成
2. **覆盖割**：针对0-1 knapsack约束
3. **团割**：针对互斥变量
4. **流覆盖割**：针对固定费用问题
5. **混合整数舍入割**：通用MIP割

1.9 本章小结

本章介绍了规划优化的核心算法：

表 1.2: 规划优化算法对比

算法	适用问题	求解特点	复杂度
线性规划	连续线性	多项式/实践高效	多项式
整数规划	离散线性	分支定界/割平面	NP-hard
非线性规划	连续非线性	迭代局部搜索	依赖问题
多目标规划	多目标	Pareto前沿	高
动态规划	多阶段决策	递推精确求解	指数（状态空间）

这些算法构成了数学建模优化的基础，后续章节的智能启发式算法往往是在这些经典方法基础上的改进或补充。

Part II

智能启发式优化算法

Chapter 2

智能启发式优化算法

智能启发式优化算法（也称为元启发式算法、进化算法或群体智能算法）是一类受自然界启发的全局优化方法。它们不依赖问题的数学性质（如梯度、凸性），能够处理复杂的非线性、非凸、离散优化问题。

2.1 遗传算法 (Genetic Algorithm, GA)

2.1.1 基本概念

遗传算法模拟生物进化过程，通过选择、交叉、变异等操作，使种群逐步进化到更优状态。

核心术语：

- **个体 (Individual)**：问题的一个候选解，通常编码为染色体
- **种群 (Population)**：多个个体的集合
- **适应度 (Fitness)**：衡量个体优劣的指标，由目标函数计算
- **基因 (Gene)**：染色体的一个位置，对应一个决策变量
- **选择 (Selection)**：根据适应度选择优良个体繁殖后代
- **交叉 (Crossover)**：两个父代个体交换基因片段产生子代
- **变异 (Mutation)**：随机改变个体的某些基因

2.1.2 工作原理

遗传算法基于达尔文进化论的三个核心原理：

1. **自然选择**：适应度高的个体有更高的生存和繁殖机会
2. **遗传**：子代继承父代的优良特性
3. **变异**：引入新的遗传多样性，避免早熟收敛

2.1.3 编码方式

1. **二进制编码**：将变量转换为二进制串

$$x = a + \frac{b-a}{2^L - 1} \sum_{i=1}^{L} b_i 2^{i-1} \tag{2.1}$$

其中 $[a, b]$ 为变量范围，L 为编码长度。

2. **实数编码**：直接使用实数值作为基因，适合连续优化。

3. **排列编码**：用于排序、TSP等问题，如 $[3, 1, 4, 2]$ 表示访问顺序。

4. **树形编码**：用于遗传编程，表示程序结构。

2.1.4 遗传算法伪代码

Algorithm 2.1 标准遗传算法

输入: 种群大小 N，最大代数 G_{max}，交叉概率 p_c，变异概率 p_m

输出: 最优解 $\mathbf{x}^*$

初始化: 随机生成初始种群 $P(0) = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$

计算每个个体的适应度 $f(\mathbf{x}_i)$
for $gen = 1$ to G_{max} **do**
    选择：根据适应度从 $P(gen-1)$ 中选择 N 个个体形成交配池
    （常用轮盘赌选择或锦标赛选择）
    交叉：对交配池中的个体两两配对
    for 每对父代 $(\mathbf{p}_1, \mathbf{p}_2)$ **do**
        if $\text{random}(0,1) < p_c$ **then**
            执行交叉操作，产生两个子代 $(\mathbf{c}_1, \mathbf{c}_2)$
        else
            $(\mathbf{c}_1, \mathbf{c}_2) \leftarrow (\mathbf{p}_1, \mathbf{p}_2)$
        end if
    end for
    变异：对每个子代个体
    for 每个子代 $\mathbf{c}_i$ **do**
        for 每个基因位 j **do**
            if $\text{random}(0,1) < p_m$ **then**
                随机改变 $\mathbf{c}_i$ 的第 j 个基因
            end if
        end for
    end for
    形成新一代种群 $P(gen)$
    精英保留（可选）：用上一代最优个体替换当代最差个体
    计算新种群的适应度
    记录当前最优解
end for
return 历史最优解 $\mathbf{x}^*$

2.1.5 选择算子

1. 轮盘赌选择 (Roulette Wheel Selection) 个体被选中的概率与其适应度成正比：

$$P(\mathbf{x}_i) = \frac{f(\mathbf{x}_i)}{\sum_{j=1}^{N} f(\mathbf{x}_j)} \tag{2.2}$$

2. 锦标赛选择 (Tournament Selection) 随机选择 k 个个体，选取其中适应度最高的。k 越大，选择压力越大。

3. 排名选择 (Rank Selection) 按适应度排序，选择概率基于排名而非实际适应度值，避免超级个体主导。

2.1.6 交叉算子

1. 单点交叉

父代1: `1101|0110`　　父代2: `0110|1001`
子代1: `1101|1001`　　子代2: `0110|0110`

2. 多点交叉 选择多个交叉点进行基因交换。

3. 均匀交叉 每个基因位独立决定来自哪个父代。

4. 算术交叉（实数编码）

$$\mathbf{c}_1 = \alpha \mathbf{p}_1 + (1-\alpha)\mathbf{p}_2 \tag{2.3}$$

5. Order Crossover (OX)（排列编码）保持子序列的相对顺序，用于TSP等问题。

2.1.7 变异算子

1. 位翻转变异（二进制）随机选择一个或多个位，0变1，1变0。

2. 高斯变异（实数编码）

$$x_i' = x_i + N(0, \sigma^2) \tag{2.4}$$

3. 交换变异（排列编码）随机交换两个位置的元素。

4. 逆序变异 随机选择一个子序列并反转。

2.1.8 算法变体

1. **精英遗传算法：**保留每代最优个体不参与交叉变异，直接进入下一代。

2. **自适应遗传算法：**根据种群多样性动态调整 p_c 和 p_m。

3. **并行遗传算法：**
   - 主从式：主节点协调，从节点评估适应度
   - 粗粒度：多个子种群，定期迁移个体
   - 细粒度：每个个体在一个网格上，只与邻居交互
4. **混合遗传算法：**结合局部搜索（如模拟退火、梯度下降）进行个体精化。
5. **多目标遗传算法：**NSGA-II、SPEA2等（见多目标优化章节）。
6. **量子遗传算法：**用量子比特表示个体，具有叠加态特性。

2.1.9 优缺点分析

优点：

- 不依赖梯度信息，适用于黑箱优化
- 全局搜索能力强，不易陷入局部最优
- 可以处理离散、连续、混合变量
- 易于并行化
- 适用范围广，几乎可用于任何优化问题

缺点：

- 收敛速度较慢，需要大量函数评估
- 参数设置（种群大小、交叉/变异概率）影响性能
- 不能保证找到全局最优解
- 对于高精度要求的连续优化，后期搜索效率低
- 早熟收敛风险（种群多样性丧失）

2.1.10 应用领域

函数优化、组合优化、机器学习（特征选择、神经网络训练）、工程设计优化、调度问题、路径规划、游戏策略优化等。

2.1.11 注意事项

1. **参数调优**：典型设置 $N = 50 \sim 200$, $p_c = 0.6 \sim 0.9$, $p_m = 0.01 \sim 0.1$
2. **编码设计**：编码应能覆盖可行域，且便于交叉变异操作
3. **适应度缩放**：避免早期超级个体过早主导种群
4. **多样性维护**：监测种群多样性，必要时重启或增加变异
5. **约束处理**：使用罚函数法或可行规则处理约束
6. **终止条件**：最大代数、适应度阈值、停滞代数等

2.2 粒子群优化算法 (Particle Swarm Optimization, PSO)

2.2.1 基本概念

PSO模拟鸟群觅食行为，每个粒子代表解空间中的一个候选解，通过跟踪个体历史最优和群体历史最优来更新自己的位置和速度。

核心概念：

- **粒子 (Particle)**：解空间中的一个点，有位置和速度
- **个体最优 (pBest)**：粒子自身历史最佳位置
- **全局最优 (gBest)**：整个 swarm 的历史最佳位置
- **惯性权重**：控制之前速度的影响程度
- **学习因子**：控制向 pBest 和 gBest 学习的程度

2.2.2 工作原理

每个粒子在解空间中飞行，其速度受三部分影响：

1. **惯性**：保持之前的运动方向
2. **认知成分**：向自身历史最优位置移动
3. **社会成分**：向群体历史最优位置移动

2.2.3 PSO算法伪代码

Algorithm 2.2 标准粒子群优化算法

输入: 种群大小 N，最大迭代次数 T_{max}，惯性权重 w，学习因子 c_1, c_2

输出: 最优解 $\mathbf{x}^*$

初始化: 随机初始化 N 个粒子的位置 $\mathbf{x}_i$ 和速度 $\mathbf{v}_i$

计算每个粒子的适应度 $f(\mathbf{x}_i)$
初始化个体最优：$\mathbf{p}_i \leftarrow \mathbf{x}_i$
初始化全局最优：$\mathbf{g} \leftarrow \arg\min_{\mathbf{x}_i} f(\mathbf{x}_i)$
for $t = 1$ to T_{max} **do**
    for 每个粒子 $i = 1$ to N **do**
        for 每个维度 $d = 1$ to D **do**
            生成随机数 $r_1, r_2 \sim U(0, 1)$
            更新速度：
            $v_{i,d} \leftarrow w \cdot v_{i,d} + c_1 \cdot r_1 \cdot (p_{i,d} - x_{i,d}) + c_2 \cdot r_2 \cdot (g_d - x_{i,d})$
            速度钳制：$v_{i,d} \leftarrow \text{clip}(v_{i,d}, -v_{max}, v_{max})$
            更新位置：
            $x_{i,d} \leftarrow x_{i,d} + v_{i,d}$
            边界处理：若超出边界则反弹或重置
        end for
        计算新位置的适应度 $f(\mathbf{x}_i)$
        if $f(\mathbf{x}_i) < f(\mathbf{p}_i)$ **then**
            更新个体最优：$\mathbf{p}_i \leftarrow \mathbf{x}_i$
            if $f(\mathbf{x}_i) < f(\mathbf{g})$ **then**
                更新全局最优：$\mathbf{g} \leftarrow \mathbf{x}_i$
            end if
        end if
    end for
    （可选）更新惯性权重 w
end for
return 最优解 $\mathbf{x}^* = \mathbf{g}$

2.2.4 速度更新公式详解

标准PSO的速度更新公式：

$$\mathbf{v}_i(t+1) = w\mathbf{v}_i(t) + c_1\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i(t)) + c_2\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i(t)) \tag{2.5}$$

其中 $\odot$ 表示逐元素乘法。

参数说明：

- w：惯性权重，典型值 $0.4 \sim 0.9$，控制探索与开发的平衡
- c_1：认知学习因子，典型值 $1.5 \sim 2.0$
- c_2：社会学习因子，典型值 $1.5 \sim 2.0$
- $\mathbf{r}_1, \mathbf{r}_2$：随机向量，各分量服从 $U(0,1)$

2.2.5 算法变体

1. **惯性权重递减PSO：**

$$w(t) = w_{max} - \frac{w_{max} - w_{min}}{T_{max}} \cdot t \tag{2.6}$$

早期大权重利于全局探索，后期小权重利于局部开发。

2. **压缩因子PSO (Constriction PSO)：**

$$\mathbf{v}_i = \chi[\mathbf{v}_i + c_1\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i)] \tag{2.7}$$

其中 χ 为压缩因子，保证收敛。

3. **拓扑结构PSO：**
   - 全局版 (gbest)：所有粒子共享同一个全局最优
   - 局部版 (lbest)：每个粒子只与邻居共享最优
   - 环形、星形、方形等拓扑结构
4. **完全信息PSO (FIPS)**：每个粒子从所有邻居学习。
5. **多阶段PSO**：不同阶段使用不同参数策略。
6. **离散PSO**：用于组合优化，位置更新采用概率方式。
7. **多目标PSO (MOPSO)**：维护Pareto前沿档案，引导粒子搜索。

2.2.6 优缺点分析

优点：

- 实现简单，参数少

- 收敛速度快（相比GA）
- 记忆功能：保留历史最优信息
- 适合连续优化问题
- 易于并行化

缺点：

- 容易早熟收敛，陷入局部最优
- 对于高维复杂问题性能下降
- 缺乏严格的收敛理论保证
- 对参数敏感
- 不适合离散/组合优化（需要特殊处理）

2.2.7 应用领域

函数优化、神经网络训练、电力系统优化、天线设计、机器人路径规划、图像处理、数据挖掘等。

2.2.8 注意事项

1. **参数设置**：推荐 $w = 0.7298$, $c_1 = c_2 = 1.49618$（经验值）
2. **速度限制**：设置 v_{max} 防止粒子飞离搜索空间
3. **边界处理**：吸收墙、反射墙、周期性边界等
4. **种群大小**：通常 $20 \sim 50$ 个粒子
5. **多样性监测**：当种群聚集时，可重新初始化部分粒子
6. **混合策略**：结合局部搜索提高精度

2.3 模拟退火算法 (Simulated Annealing, SA)

2.3.1 基本概念

模拟退火算法模拟金属退火过程：高温下原子可以自由运动，缓慢降温过程中原子逐渐排列成能量最低的稳定结构。

物理类比：

- 温度 T：控制接受劣解的概率
- 能量 E：对应目标函数值 $f(\mathbf{x})$
- 退火 schedule：温度降低策略

2.3.2 工作原理

SA的核心是Metropolis准则：以一定概率接受劣解，从而跳出局部最优。接受概率为：

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta E < 0 \\ e^{-\Delta E/T} & \text{if } \Delta E \geq 0 \end{cases} \tag{2.8}$$

其中 $\Delta E = f(\mathbf{x}_{new}) - f(\mathbf{x}_{current})$。

2.3.3 模拟退火伪代码

Algorithm 2.3 模拟退火算法

输入: 初始温度 T_0，终止温度 T_f，降温系数 α，每温步迭代次数 L
输出: 最优解 $\mathbf{x}^*$
初始化: 随机生成初始解 $\mathbf{x}_{current}$

计算 $f(\mathbf{x}_{current})$
$\mathbf{x}^* \leftarrow \mathbf{x}_{current}$，$f^* \leftarrow f(\mathbf{x}_{current})$
$T \leftarrow T_0$
while $T > T_f$ **do**
  for $l = 1$ to L **do**
    在 $\mathbf{x}_{current}$ 邻域内生成新解 $\mathbf{x}_{new}$
    计算 $\Delta f = f(\mathbf{x}_{new}) - f(\mathbf{x}_{current})$
    if $\Delta f < 0$ 或 $\text{random}(0,1) < e^{-\Delta f/T}$ **then**
      接受新解：$\mathbf{x}_{current} \leftarrow \mathbf{x}_{new}$
      if $f(\mathbf{x}_{new}) < f^*$ **then**
        更新最优：$\mathbf{x}^* \leftarrow \mathbf{x}_{new}$，$f^* \leftarrow f(\mathbf{x}_{new})$
      end if
    end if
  end for
  降温：$T \leftarrow \alpha \cdot T$
end while
return 最优解 $\mathbf{x}^*$

2.3.4 关键组件

1. 邻域结构 定义如何从当前解生成新解：

- 连续优化：$\mathbf{x}_{new} = \mathbf{x}_{current} + \delta$，$\delta \sim N(0, \sigma^2)$
- 组合优化：交换、反转、插入等操作
- 邻域大小随温度调整

2. 退火策略

- **几何降温**：$T_{k+1} = \alpha T_k$，$\alpha \in [0.8, 0.99]$
- **线性降温**：$T_{k+1} = T_k - \Delta T$
- **对数降温**：$T_k = T_0 / \ln(k+1)$，理论上保证收敛但太慢
- **自适应降温**：根据接受率调整降温速度

3. 初始温度设定

- 经验法：T_0 使得初始接受率约 $80\% \sim 90\%$
- 统计法：采样若干解，令 $T_0 = \sigma_f / \ln(1/\chi)$

2.3.5 算法变体

1. **快速模拟退火 (Fast SA)**：使用柯西分布生成新解，收敛更快。
2. **超快模拟退火 (Very Fast SA)**：更激进的搜索策略。
3. **自适应模拟退火**：根据搜索进程动态调整参数。
4. **并行模拟退火**：
   - 独立运行多个SA，最后取最优
   - 周期性交换信息
5. **混合模拟退火**：结合其他算法（如TS、GA）的优点。

2.3.6 优缺点分析

优点：

- 实现简单，只需少量参数
- 能以概率1渐近收敛到全局最优
- 适用于各种类型的问题
- 内存需求小
- 灵活性强，易于与其他算法结合

缺点：

- 收敛速度慢，需要大量迭代
- 参数 tuning 需要经验
- 没有记忆机制，可能重复搜索
- 对于高维问题效率低
- 难以确定合适的退火 schedule

2.3.7 应用领域

TSP、VLSI布局、作业车间调度、图着色、神经网络训练、图像分割、蛋白质折叠等。

2.3.8 注意事项

1. **温度参数**：T_0 要足够大，T_f 要足够小
2. **降温速度**：太快易陷入局部最优，太慢浪费时间
3. **邻域设计**：应能覆盖整个可行域
4. **每温步迭代次数**：确保达到热平衡
5. **重启策略**：长时间无改进时可重新加热

2.4 蚁群算法 (Ant Colony Optimization, ACO)

2.4.1 基本概念

ACO模拟蚂蚁觅食行为：蚂蚁在路径上留下信息素，后续蚂蚁倾向于选择信息素浓度高的路径，形成正反馈。

核心机制：

- **信息素 (Pheromone)**：路径上的标记，引导后续蚂蚁
- **启发式信息**：问题的先验知识（如距离的倒数）
- **挥发**：信息素随时间逐渐减少，避免过早收敛
- **增强**：优质路径获得更多信息素

2.4.2 工作原理（以TSP为例）

蚂蚁构建解的过程：

1. 每只蚂蚁从随机城市出发
2. 根据转移概率选择下一个未访问城市
3. 完成所有城市访问后，得到一条回路
4. 根据回路长度更新信息素

转移概率：

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in \text{allowed}_k} [\tau_{il}]^\alpha [\eta_{il}]^\beta} \tag{2.9}$$

其中：

- τ_{ij}：边 (i, j) 上的信息素浓度
- $\eta_{ij} = 1/d_{ij}$：启发式信息（距离倒数）
- α：信息素重要性参数
- β：启发式信息重要性参数

2.4.3 蚁群算法伪代码

Algorithm 2.4 蚁群系统 (Ant Colony System)

输入: 蚂蚁数量 m，最大迭代次数 N_{max}，信息素重要程度 α, β，挥发系数 ρ

输出: 最优路径 π^*

初始化: 初始化信息素矩阵 $\tau_{ij} = \tau_0$

$\pi^* \leftarrow \emptyset$，$L^* \leftarrow \infty$
for $iter = 1$ to N_{max} **do**
    for 每只蚂蚁 $k = 1$ to m **do**
        随机选择起始城市
        初始化未访问城市集合
        while 存在未访问城市 **do**
            当前在城市 i
            根据转移概率 p_{ij}^k 选择下一个城市 j
            移动到城市 j，标记为已访问
        end while
        返回起始城市，完成回路
        计算回路长度 L_k
        if $L_k < L^*$ **then**
            $\pi^* \leftarrow$ 蚂蚁 k 的路径
            $L^* \leftarrow L_k$
        end if
    end for
    信息素挥发:
    $\tau_{ij} \leftarrow (1-\rho)\tau_{ij}$，对所有边 (i,j)
    信息素增强:
    for 每只蚂蚁 k **do**
        for 路径 π_k 中的每条边 (i,j) **do**
            $\tau_{ij} \leftarrow \tau_{ij} + Q/L_k$
        end for
    end for
    （可选）**精英策略:** 额外增强最优路径 π^* 的信息素
end for
return 最优路径 π^*

2.4.4 算法变体

1. **Ant System (AS)**：最早的ACO版本，所有蚂蚁都贡献信息素。
2. **Elitist Ant System (EAS)**：最优解的蚂蚁额外贡献信息素。
3. **Max-Min Ant System (MMAS)**：
   - 只有最优蚂蚁更新信息素
   - 信息素浓度限制在 $[\tau_{min}, \tau_{max}]$ 范围内
4. **Ant Colony System (ACS)**：
   - 引入局部信息素更新（构建解时）
   - 只有最优蚂蚁进行全局更新
5. **Rank-Based Ant System (RAS)**：按路径质量排名，排名靠前的蚂蚁贡献更多信息素。
6. **Continuous Orthogonal Ant Colony (COAC)**：用于连续优化。

2.4.5 优缺点分析

优点：

- 正反馈机制能快速找到较优解
- 分布式计算，易于并行化
- 鲁棒性强，对参数不敏感
- 适合组合优化问题
- 能找到多个较优解

缺点：

- 收敛速度相对较慢
- 早期容易出现停滞现象
- 参数较多，需要调节
- 理论基础不如其他算法完善
- 对于连续优化需要特殊处理

2.4.6 应用领域

旅行商问题、车辆路径问题、网络路由、任务分配、图着色、数据挖掘、机器人路径规划等。

2.4.7 注意事项

1. **参数设置**：典型值 $\alpha = 1$, $\beta = 2 \sim 5$, $\rho = 0.1 \sim 0.5$
2. **蚂蚁数量**：通常为城市数的 $1 \sim 2$ 倍
3. **初始信息素**：$\tau_0 = m/L_{nn}$，L_{nn} 为最近邻启发式解的长度
4. **候选列表**：限制可选城市数量，加速搜索
5. **局部搜索**：结合2-opt、3-opt等局部优化
6. **信息素上下界**：防止某些路径信息素过高或过低

2.5 禁忌搜索 (Tabu Search, TS)

2.5.1 基本概念

禁忌搜索通过维护一个禁忌表来避免循环搜索，允许接受劣解以跳出局部最优。

核心概念：

- **禁忌表 (Tabu List)**：记录近期访问过的解或移动
- **禁忌期限 (Tabu Tenure)**：禁止某移动的时间长度
- **渴望准则 (Aspiration Criterion)**：即使移动被禁忌，如果足够好也可以接受
- **邻域**：当前解附近的一组解

2.5.2 工作原理

TS从一个初始解开始，在每步迭代中：

1. 生成当前解的邻域
2. 排除禁忌的移动
3. 选择最好的非禁忌移动（或满足渴望准则的移动）

4. 更新禁忌表

5. 更新最优解

2.5.3 禁忌搜索伪代码

Algorithm 2.5 禁忌搜索算法

输入: 初始解 $\mathbf{x}_0$，禁忌表长度 L，最大迭代次数 N_{max}

输出: 最优解 $\mathbf{x}^*$

初始化: $\mathbf{x}_{current} \leftarrow \mathbf{x}_0$

$\mathbf{x}^* \leftarrow \mathbf{x}_0$，$f^* \leftarrow f(\mathbf{x}_0)$
初始化禁忌表 $T = \emptyset$
for $iter = 1$ to N_{max} **do**
    生成 $\mathbf{x}_{current}$ 的邻域 $N(\mathbf{x}_{current})$
    从邻域中选择最佳候选解 $\mathbf{x}_{candidate}$:
    $\mathbf{x}_{candidate} = \arg\min_{\mathbf{x} \in N(\mathbf{x}_{current})} \{f(\mathbf{x}) : \text{move not in } T \text{ or meets aspiration}\}$
    执行移动：$\mathbf{x}_{current} \leftarrow \mathbf{x}_{candidate}$
    更新禁忌表:
    将刚执行的移动加入 T
    移除过期的移动（超过禁忌期限）
    if $f(\mathbf{x}_{current}) < f^*$ **then**
        $\mathbf{x}^* \leftarrow \mathbf{x}_{current}$
        $f^* \leftarrow f(\mathbf{x}_{current})$
    end if
end for
return 最优解 $\mathbf{x}^*$

2.5.4 关键组件

1. 禁忌对象

- **解禁忌**：将整个解加入禁忌表（占用空间大）
- **移动禁忌**：记录执行的移动类型（更常用）
- **属性禁忌**：记录解的某些属性变化

2. 禁忌期限

- 固定长度：$L = 7$ 或 $L = \sqrt{n}$

- 动态变化：$L \in [L_{min}, L_{max}]$ 随机选择
- 自适应：根据搜索状态调整

3. 渴望准则 常见策略：如果移动产生的解优于历史最优，则无视禁忌。
4. 邻域结构 根据问题设计，如TSP中的2-opt、swap、insert等。

2.5.5 算法变体

1. **严格禁忌搜索**：绝对不允许禁忌移动。
2. **反应禁忌搜索 (Reactive TS)**：根据搜索历史自动调整禁忌期限。
3. **平行禁忌搜索**：多个搜索线程并行运行。
4. **混合禁忌搜索**：与其他算法（如SA、GA）结合。
5. **超启发式禁忌搜索**：在启发式方法空间进行搜索。

2.5.6 优缺点分析

优点：

- 能有效跳出局部最优
- 避免循环搜索
- 实现相对简单
- 适合组合优化问题
- 可以找到高质量解

缺点：

- 禁忌表和邻域设计需要领域知识
- 参数（禁忌期限）影响性能
- 对于大规模问题，邻域生成成本高
- 缺乏理论收敛保证
- 可能需要较长的运行时间

2.5.7 应用领域

调度问题、图着色、二次分配、电路设计、电信网络优化、资源分配等。

2.6 差分进化算法 (Differential Evolution, DE)

2.6.1 基本概念

DE是一种基于种群的连续优化算法，通过差分变异和交叉操作生成新个体。

核心操作：

- **变异：**利用种群中个体的差分向量
- **交叉：**将变异个体与目标个体混合
- **选择：**贪婪选择，保留更好的个体

2.6.2 DE算法伪代码

Algorithm 2.6 差分进化算法 (DE/rand/1/bin)

输入: 种群大小 NP，缩放因子 F，交叉概率 CR，最大代数 G_{max}

输出: 最优解 $\mathbf{x}^*$

初始化: 随机生成初始种群 $\{\mathbf{x}_1, \ldots, \mathbf{x}_{NP}\}$

计算每个个体的适应度
for $gen = 1$ to G_{max} **do**
  for 每个个体 $i = 1$ to NP **do**
    变异：随机选择三个不同个体 $\mathbf{x}_{r1}, \mathbf{x}_{r2}, \mathbf{x}_{r3}$
    $\mathbf{v}_i = \mathbf{x}_{r1} + F \cdot (\mathbf{x}_{r2} - \mathbf{x}_{r3})$
    交叉：生成试验向量 $\mathbf{u}_i$
    for 每个维度 $j = 1$ to D **do**
      if $\text{random}(0,1) < CR$ 或 $j = j_{rand}$ **then**
        $u_{i,j} \leftarrow v_{i,j}$
      else
        $u_{i,j} \leftarrow x_{i,j}$
      end if
    end for
    选择：
    if $f(\mathbf{u}_i) \leq f(\mathbf{x}_i)$ **then**
      $\mathbf{x}_i \leftarrow \mathbf{u}_i$
    end if
  end for
end for
return 种群中最优个体 $\mathbf{x}^*$

2.6.3 DE策略表示

DE/x/y/z 表示法：

- x：变异策略（rand, best, current-to-best等）
- y：差分向量数量
- z：交叉类型（bin二项式, exp指数）

常见策略：

- DE/rand/1/bin：经典策略
- DE/best/1/bin：利用最优个体
- DE/current-to-best/1：当前个体向最优靠近
- DE/rand/2/bin：使用两个差分向量

2.6.4 优缺点分析

优点：

- 参数少，易于使用
- 收敛速度快
- 鲁棒性强
- 适合连续优化
- 不需要梯度信息

缺点：

- 对于离散问题需要修改
- 高维问题性能下降
- 后期收敛精度有限
- 对参数 F 和 CR 敏感

2.7 灰狼优化算法 (Grey Wolf Optimizer, GWO)

2.7.1 基本概念

GWO模拟灰狼的社会等级和捕猎行为。狼群分为四个等级：

- α：头狼，最优解
- β：次优解，协助 α
- δ：第三优解
- ω：其他个体，跟随前三者

2.7.2 GWO算法伪代码

Algorithm 2.7 灰狼优化算法

输入: 种群大小 N，最大迭代次数 T_{max}

输出: 最优解 $\mathbf{x}^*$

初始化: 随机生成灰狼种群 $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$

确定 α, β, δ（前三个最优个体）
for $t = 1$ to T_{max} **do**
  for 每个个体 i **do**
    计算收敛因子：$a = 2 - 2t/T_{max}$
    生成随机向量：$\mathbf{A} = 2a \cdot \mathbf{r}_1 - a$
    $\mathbf{C} = 2 \cdot \mathbf{r}_2$
    计算到三头的距离：
    $\mathbf{D}_\alpha = |\mathbf{C}_1 \cdot \mathbf{x}_\alpha - \mathbf{x}_i|$
    $\mathbf{D}_\beta = |\mathbf{C}_2 \cdot \mathbf{x}_\beta - \mathbf{x}_i|$
    $\mathbf{D}_\delta = |\mathbf{C}_3 \cdot \mathbf{x}_\delta - \mathbf{x}_i|$
    更新位置：
    $\mathbf{x}_1 = \mathbf{x}_\alpha - \mathbf{A}_1 \cdot \mathbf{D}_\alpha$
    $\mathbf{x}_2 = \mathbf{x}_\beta - \mathbf{A}_2 \cdot \mathbf{D}_\beta$
    $\mathbf{x}_3 = \mathbf{x}_\delta - \mathbf{A}_3 \cdot \mathbf{D}_\delta$
    $\mathbf{x}_i(t+1) = (\mathbf{x}_1 + \mathbf{x}_2 + \mathbf{x}_3)/3$
  end for
  更新 α, β, δ
end for
return $\mathbf{x}^* = \mathbf{x}_\alpha$

2.7.3 优缺点

优点：实现简单、参数少、收敛快；**缺点**：易早熟、高维性能差。

2.8 其他智能优化算法概述

2.8.1 麻雀搜索算法 (Sparrow Search Algorithm, SSA)

模拟麻雀的觅食和反捕食行为，分为发现者（探索）和加入者（开发）。

2.8.2 鲸鱼优化算法 (Whale Optimization Algorithm, WOA)

模拟座头鲸的气泡网捕食策略，包括收缩包围和螺旋更新两种机制。

2.8.3 人工鱼群算法 (Artificial Fish Swarm Algorithm, AFSA)

模拟鱼群的觅食、聚群、追尾行为，具有良好的全局搜索能力。

2.8.4 萤火虫算法 (Firefly Algorithm, FA)

基于萤火虫的发光吸引行为，亮度（适应度）高的萤火虫吸引其他萤火虫。

2.8.5 人工蜂群算法 (Artificial Bee Colony, ABC)

模拟蜜蜂采蜜，分为雇佣蜂、观察蜂和侦查蜂三种角色。

2.8.6 布谷鸟搜索 (Cuckoo Search, CS)

基于布谷鸟的巢寄生行为和莱维飞行机制。

2.9 智能优化算法对比与选择

表 2.1: 主要智能优化算法特性对比

算法	参数数量	收敛速度	全局搜索	适用问题
GA	中	慢	强	通用
PSO	少	快	中	连续
SA	少	慢	强	通用
ACO	多	中	强	组合
TS	中	中	中	组合
DE	少	快	强	连续
GWO	少	快	中	连续

选择建议：

- 连续优化：优先尝试 PSO、DE、GWO
- 组合优化：优先尝试 ACO、TS、GA
- 黑箱优化：GA、SA 适用性广
- 高维问题：DE、PSO 表现较好
- 需要精确解：混合局部搜索

2.10 本章小结

本章介绍了11种主要的智能启发式优化算法。这些算法各有特点，在实际应用中应根据问题特性选择合适的算法，并可考虑混合多种算法以获得更好的性能。

Part III

时序与预测类算法

Chapter 3

时序与预测类算法

时间序列预测是数学建模中的核心任务之一，广泛应用于经济预测、气象预报、销售预测、股票分析等领域。本章系统介绍从经典统计方法到现代机器学习的多种预测技术。

3.1 回归分析基础

3.1.1 一元线性回归

对于两个变量 x（自变量）和 y（因变量），假设它们之间存在线性关系：

$$y = \beta_0 + \beta_1 x + \varepsilon \tag{3.1}$$

其中 $\varepsilon \sim N(0, \sigma^2)$ 为随机误差项。

参数估计（最小二乘法）：

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \tag{3.2}$$

统计推断：

- **决定系数：** $R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$
- β_1 **的 t 检验：** $t = \frac{\hat{\beta}_1}{s_{\hat{\beta}_1}} \sim t(n-2)$
- **置信区间：** $\hat{\beta}_1 \pm t_{\alpha/2, n-2} \cdot s_{\hat{\beta}_1}$

3.1.2 多元线性回归

对于 p 个自变量：

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon \tag{3.3}$$

矩阵形式：$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$

OLS估计：

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{3.4}$$

性质：

- 无偏性：$E[\hat{\boldsymbol{\beta}}] = \boldsymbol{\beta}$
- 协方差矩阵：$\text{Var}(\hat{\boldsymbol{\beta}}) = \sigma^2(\mathbf{X}^T\mathbf{X})^{-1}$
- Gauss-Markov定理：在所有线性无偏估计中，OLS具有最小方差

3.1.3 逐步回归算法

Algorithm 3.1 逐步回归（双向）

输入: 候选变量集合 X_{all}，显著性水平 $\alpha_{in}, \alpha_{out}$

输出: 选定的变量子集和回归模型

初始化: $X_{selected} \leftarrow \emptyset$, $X_{remaining} \leftarrow X_{all}$

repeat
  向前步骤：
  for 每个变量 $x_j \in X_{remaining}$ **do**
    拟合包含 x_j 的模型，计算 p 值 p_j
  end for
  选择 $p_{min} = \min p_j$ 对应的变量 x_{best}
  if $p_{min} < \alpha_{in}$ **then**
    $X_{selected} \leftarrow X_{selected} \cup \{x_{best}\}$
    $X_{remaining} \leftarrow X_{remaining} \setminus \{x_{best}\}$
  end if
  向后步骤：
  for 每个变量 $x_k \in X_{selected}$ **do**
    计算移除 x_k 后的 p 值 p_k
  end for
  选择 $p_{max} = \max p_k$ 对应的变量 x_{worst}
  if $p_{max} > \alpha_{out}$ **then**
    $X_{selected} \leftarrow X_{selected} \setminus \{x_{worst}\}$
    $X_{remaining} \leftarrow X_{remaining} \cup \{x_{worst}\}$
  end if
until 没有变量加入或移除
return 最终模型

3.1.4 回归诊断

常见问题及检测：

1. **多重共线性：**

   - VIF（方差膨胀因子）：$VIF_j = \frac{1}{1-R_j^2}$，VIF¿10表示严重共线性
   - 条件数：$\kappa = \sqrt{\frac{\lambda_{max}}{\lambda_{min}}}$，$\kappa > 1000$ 表示共线性

2. **异方差性：**

   - Breusch-Pagan检验
   - White检验
   - 残差图观察

3. **自相关：**

   - Durbin-Watson统计量：$DW = \frac{\sum_{t=2}^{n}(e_t - e_{t-1})^2}{\sum_{t=1}^{n} e_t^2}$
   - $DW \approx 2$ 表示无自相关

4. **异常值检测：**

   - 学生化残差：$|r_i| > 3$ 可能为异常值
   - Cook距离：$D_i > 4/n$ 表示强影响点

3.2 ARIMA模型详解

3.2.1 平稳性检验

ADF检验（Augmented Dickey-Fuller Test）：
回归方程：

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^{p} \delta_i \Delta y_{t-i} + \varepsilon_t \tag{3.5}$$

假设检验：

- $H_0 : \gamma = 0$（存在单位根，非平稳）
- $H_1 : \gamma < 0$（平稳）

若 p 值 ¡ 0.05，拒绝原假设，认为序列平稳。

3.2.2 模型识别

ACF和PACF模式：

表 3.1: ARMA模型的ACF和PACF特征

模型	ACF	PACF
AR(p)	拖尾	p阶截尾
MA(q)	q阶截尾	拖尾
ARMA(p,q)	拖尾	拖尾

信息准则定阶：

$$AIC = n\ln(\hat{\sigma}^2) + 2k \tag{3.6}$$

$$BIC = n\ln(\hat{\sigma}^2) + k\ln n \tag{3.7}$$

选择使AIC或BIC最小的 (p,q) 组合。

3.2.3 参数估计

最大似然估计：

对于ARMA模型，对数似然函数为：

$$\ln L = -\frac{n}{2}\ln(2\pi) - \frac{n}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{t=1}^{n}\varepsilon_t^2 \tag{3.8}$$

通过数值优化（如Newton-Raphson）求解。

3.2.4 模型诊断

1. **残差白噪声检验**（Ljung-Box test）：

$$Q = n(n+2)\sum_{k=1}^{K}\frac{\hat{\rho}_k^2}{n-k} \sim \chi^2(K-p-q) \tag{3.9}$$

若 p 值 ¿ 0.05，接受残差为白噪声的假设。

2. **残差正态性检验**：Q-Q图、Jarque-Bera检验

3. **参数显著性**：t检验，p 值 ¡ 0.05

3.2.5 预测

h步超前预测：

$$\hat{y}_{T+h|T} = E[y_{T+h}|y_1, \ldots, y_T] \tag{3.10}$$

预测区间：

$$\hat{y}_{T+h|T} \pm z_{\alpha/2}\sqrt{Var(e_{T+h})} \tag{3.11}$$

3.3 指数平滑法详解

3.3.1 简单指数平滑

适用于无趋势、无季节性的平稳序列：

$$\ell_t = \alpha y_t + (1-\alpha)\ell_{t-1} \tag{3.12}$$

预测：$\hat{y}_{t+h|t} = \ell_t$

参数选择： 最小化 SSE，或使用 AICc 准则。

3.3.2 Holt线性趋势法

$$\text{水平：} \quad \ell_t = \alpha y_t + (1-\alpha)(\ell_{t-1} + b_{t-1}) \tag{3.13}$$

$$\text{趋势：} \quad b_t = \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1} \tag{3.14}$$

$$\text{预测：} \quad \hat{y}_{t+h|t} = \ell_t + hb_t \tag{3.15}$$

3.3.3 Holt-Winters季节性方法

加法模型（适合季节波动幅度恒定）：

$$\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1}) \tag{3.16}$$

$$b_t = \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1} \tag{3.17}$$

$$s_t = \gamma(y_t - \ell_{t-1} - b_{t-1}) + (1-\gamma)s_{t-m} \tag{3.18}$$

$$\hat{y}_{t+h|t} = \ell_t + hb_t + s_{t+h-m(k+1)} \tag{3.19}$$

乘法模型（适合季节波动幅度随水平变化）：

$$\ell_t = \alpha \frac{y_t}{s_{t-m}} + (1-\alpha)(\ell_{t-1} + b_{t-1}) \tag{3.20}$$

$$b_t = \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1} \tag{3.21}$$

$$s_t = \gamma \frac{y_t}{\ell_{t-1} + b_{t-1}} + (1-\gamma)s_{t-m} \tag{3.22}$$

$$\hat{y}_{t+h|t} = (\ell_t + hb_t)s_{t+h-m(k+1)} \tag{3.23}$$

其中 m 为季节周期，$k = \lfloor (h-1)/m \rfloor$。

3.4 灰色预测 GM(1,1) 详解

3.4.1 适用条件

- 小样本：通常 $n \geq 4$ 即可建模
- 贫信息：不需要知道概率分布
- 短期预测效果较好
- 数据应具有指数规律

3.4.2 建模步骤详解

Algorithm 3.2 GM(1,1)完整建模流程

输入: 原始序列 $X^{(0)} = \{x^{(0)}(1), x^{(0)}(2), \ldots, x^{(0)}(n)\}$

输出: 预测值及精度检验结果

Step 1: 级比检验

计算级比：$\lambda(k) = \frac{x^{(0)}(k-1)}{x^{(0)}(k)}, \quad k = 2, 3, \ldots, n$

检查：$\lambda(k) \in (e^{-\frac{2}{n+1}}, e^{\frac{2}{n+1}})$

if 不满足 **then**

　　进行平移变换：$x^{(0)}(k) \leftarrow x^{(0)}(k) + c$

end if

Step 2: 累加生成 (1-AGO)

$x^{(1)}(k) = \sum_{i=1}^{k} x^{(0)}(i), \quad k = 1, 2, \ldots, n$

Step 3: 紧邻均值生成

$z^{(1)}(k) = 0.5[x^{(1)}(k) + x^{(1)}(k-1)], \quad k = 2, 3, \ldots, n$

Step 4: 建立灰微分方程

$\frac{dx^{(1)}}{dt} + ax^{(1)} = b$

Step 5: 最小二乘估计

$$B = \begin{bmatrix} -z^{(1)}(2) & 1 \\ -z^{(1)}(3) & 1 \\ \vdots & \vdots \\ -z^{(1)}(n) & 1 \end{bmatrix}, \quad Y = \begin{bmatrix} x^{(0)}(2) \\ x^{(0)}(3) \\ \vdots \\ x^{(0)}(n) \end{bmatrix}$$

$$\begin{bmatrix} a \\ b \end{bmatrix} = (B^T B)^{-1} B^T Y$$

Step 6: 时间响应函数

$\hat{x}^{(1)}(k+1) = \left(x^{(0)}(1) - \frac{b}{a}\right) e^{-ak} + \frac{b}{a}$

Step 7: 累减还原 (IAGO)

$\hat{x}^{(0)}(k+1) = \hat{x}^{(1)}(k+1) - \hat{x}^{(1)}(k)$

Step 8: 精度检验

计算残差：$\varepsilon(k) = x^{(0)}(k) - \hat{x}^{(0)}(k)$

相对误差：$\Delta_k = \frac{|\varepsilon(k)|}{x^{(0)}(k)} \times 100\%$

平均相对误差：$\bar{\Delta} = \frac{1}{n} \sum_{k=1}^{n} \Delta_k$

后验差检验：

$C = \frac{S_2}{S_1}$，其中 S_1 为原始序列标准差，S_2 为残差标准差

$P = P(|\varepsilon(k) - \bar{\varepsilon}| < 0.6745 S_1)$

return 预测序列及精度等级

3.4.3 精度等级标准

表 3.2: GM(1,1)精度等级

等级	平均相对误差	后验差比值C	小误差概率P
一级（优秀）	$< 1\%$	< 0.35	> 0.95
二级（良好）	$< 5\%$	< 0.50	> 0.80
三级（合格）	$< 10\%$	< 0.65	> 0.70
四级（不合格）	$> 10\%$	> 0.65	< 0.70

3.4.4 改进模型

- **新陈代谢 GM(1,1)**：不断加入新数据，剔除旧数据
- **Verhulst模型**：适合S型饱和增长
- **DGM(2,1)**：直接建模，无需累加
- **多变量 GM(1,N)**：考虑多个相关因素

3.5 神经网络预测

3.5.1 BP神经网络架构

网络结构：

- 输入层：n 个节点（特征数）
- 隐藏层：m 个节点（经验公式：$m \approx \sqrt{n+p} + \alpha$）
- 输出层：p 个节点（预测目标数）

激活函数：

- Sigmoid：$\sigma(x) = \frac{1}{1+e^{-x}}$
- ReLU：$f(x) = \max(0, x)$
- Tanh：$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

3.5.2 训练算法改进

1. **动量法：**

$$\Delta w(t) = -\eta \nabla E + \mu \Delta w(t-1) \tag{3.24}$$

2. **自适应学习率：**Adam、RMSprop等

3. **正则化：**

- L2正则化（权重衰减）
- Dropout：随机丢弃神经元
- Early stopping：验证集误差上升时停止

3.5.3 LSTM长短期记忆网络

门控机制：

$$\text{遗忘门：} \quad f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{3.25}$$

$$\text{输入门：} \quad i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{3.26}$$

$$\text{候选细胞状态：} \quad \tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{3.27}$$

$$\text{细胞状态更新：} \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{3.28}$$

$$\text{输出门：} \quad o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{3.29}$$

$$\text{隐藏状态：} \quad h_t = o_t \odot \tanh(C_t) \tag{3.30}$$

优势：

- 解决梯度消失问题
- 能捕捉长期依赖关系
- 适合长序列预测

3.6 预测模型选择指南

表 3.3: 预测方法适用场景对比

方法	数据需求	适用场景	预测 horizon
线性回归	中到大	线性趋势	短-中期
ARIMA	> 50	平稳时序	短期
指数平滑	> 20	有趋势/季节	短-中期
GM(1,1)	≥ 4	小样本指数	短期
BP神经网络	大	非线性复杂	短-中期
LSTM	很大	长期依赖	中长期

3.7 模型评估指标

$$\text{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t| \tag{3.31}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2} \tag{3.32}$$

$$\text{MAPE} = \frac{100\%}{n}\sum_{t=1}^{n}\left|\frac{y_t - \hat{y}_t}{y_t}\right| \tag{3.33}$$

$$R^2 = 1 - \frac{\sum(y_t - \hat{y}_t)^2}{\sum(y_t - \bar{y})^2} \tag{3.34}$$

Part IV

聚类算法

Chapter 4

聚类算法

聚类分析是无监督学习的核心方法，旨在发现数据内在的结构和模式。本章深入介绍各种聚类算法的原理、实现和应用。

4.1 聚类基础概念

4.1.1 相似性度量

聚类的核心是衡量样本之间的相似性或距离。

常用距离度量：

1. **欧氏距离（Euclidean Distance）：**

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{p}(x_i - y_i)^2} \tag{4.1}$$

2. **曼哈顿距离（Manhattan Distance）：**

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{p}|x_i - y_i| \tag{4.2}$$

3. **闵可夫斯基距离（Minkowski Distance）：**

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{p}|x_i - y_i|^q\right)^{1/q} \tag{4.3}$$

$q = 1$ 为曼哈顿距离，$q = 2$ 为欧氏距离。

4. **余弦相似度：**

$$\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T\mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|} \tag{4.4}$$

适合文本等高维稀疏数据。

5. **马氏距离（Mahalanobis Distance）：**

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^T\Sigma^{-1}(\mathbf{x} - \mathbf{y})} \tag{4.5}$$

考虑特征相关性，尺度不变。

4.1.2 聚类评估指标

内部指标（无真实标签）：

- **轮廓系数（Silhouette Coefficient）：**

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \tag{4.6}$$

其中 $a(i)$ 为簇内平均距离，$b(i)$ 为最近簇的平均距离。$s \in [-1, 1]$，越大越好。

- **Calinski-Harabasz指数：**

$$CH = \frac{B/(K-1)}{W/(N-K)} \tag{4.7}$$

B 为簇间离散度，W 为簇内离散度。越大表示聚类越分离紧凑。

- **Davies-Bouldin指数：**

$$DB = \frac{1}{K}\sum_{i=1}^{K} \max_{j \neq i} \frac{S_i + S_j}{M_{ij}} \tag{4.8}$$

越小越好。

外部指标（有真实标签）：

- **调整兰德指数（ARI）：** 衡量两个聚类结果的一致性
- **归一化互信息（NMI）：** 基于信息论的相似度度量
- **Fowlkes-Mallows指数：** 查准率和查全率的几何平均

4.2 K-Means聚类详解

4.2.1 算法原理

K-Means最小化簇内平方和（WCSS）：

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 \tag{4.9}$$

4.2.2 完整算法流程

Algorithm 4.1 K-Means聚类（增强版）

输入: 数据集 $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$，簇数 K，最大迭代 T

输出: 簇标签 $\{c_1, \ldots, c_n\}$，簇中心 $\{\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K\}$

Step 1: 初始化（K-Means++）
随机选择第一个中心 $\boldsymbol{\mu}_1$
for $k = 2$ to K **do**
    计算每个点到最近中心的距离：$D(\mathbf{x}_i) = \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$
    按概率 $P(\mathbf{x}_i) = \frac{D(\mathbf{x}_i)}{\sum_j D(\mathbf{x}_j)}$ 选择下一个中心
end for
Step 2: 迭代优化
for $t = 1$ to T **do**
    分配步骤：
    for $i = 1$ to n **do**
        $c_i \leftarrow \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$
    end for
    更新步骤：
    for $k = 1$ to K **do**
        if C_k 非空 **then**
            $\boldsymbol{\mu}_k \leftarrow \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$
        end if
    end for
    收敛检查：
    if 簇中心变化小于阈值 ϵ **then**

    end if
end for
Step 3: 评估
计算轮廓系数、WCSS等指标
return 簇标签、簇中心、评估指标

4.2.3 K值选择方法

1. **肘部法则（Elbow Method）：** 绘制WCSS随K变化的曲线，选择拐点。

2. **轮廓分析法：** 计算不同K值的平均轮廓系数，选择最大值。

3. **间隙统计（Gap Statistic）：**

$$Gap(K) = E[\log(WCSS_{null})] - \log(WCSS_{observed}) \tag{4.10}$$

选择使Gap最大的K。

4. **层次聚类辅助：** 通过树状图观察自然分组数量。

4.2.4 优缺点与改进

优点：

- 时间复杂度低：$O(nKT)$
- 实现简单，易于理解
- 适合大规模数据
- 保证收敛（局部最优）

缺点：

- 需预设K值
- 对初始值敏感
- 只能发现球形簇
- 对异常值敏感
- 不适合密度不均匀数据

改进变体：

- **K-Means++：**智能初始化，改善收敛
- **Mini-Batch K-Means：**小批量更新，加速大规模数据
- **二分K-Means：**克服局部最优
- **ISODATA：**自动合并和分裂簇
- **K-Medians：**用中位数替代均值，抗异常值

4.3 DBSCAN密度聚类详解

4.3.1 核心概念

定义 4.1 (DBSCAN关键定义).

- **ε-邻域：** $N_\varepsilon(\mathbf{p}) = \{\mathbf{q} \in D : dist(\mathbf{p}, \mathbf{q}) \leq \varepsilon\}$
- **核心点（*Core Point*）：** $|N_\varepsilon(\mathbf{p})| \geq MinPts$
- **直接密度可达：** $\mathbf{q} \in N_\varepsilon(\mathbf{p})$ 且 $\mathbf{p}$ 是核心点
- **密度可达：** 存在链 $\mathbf{p}_1, \ldots, \mathbf{p}_n$，$\mathbf{p}_{i+1}$ 从 $\mathbf{p}_i$ 直接密度可达
- **密度相连：** 两点都从某核心点密度可达

4.3.2 参数选择

1. ε 的选择：

绘制K-距离图（K-dist graph）：

1. 对每个点，计算到第K个最近邻居的距离
2. 排序后绘制曲线
3. 选择"肘部"对应的距离作为 ε

2. MinPts 的选择：

- 经验规则：$MinPts \geq d + 1$（d 为维度）
- 通常取 4-10
- 噪声多时取较大值

4.3.3 算法复杂度

- 朴素实现：$O(n^2)$
- 使用空间索引（如R-tree）：$O(n \log n)$
- 高维数据：索引效果下降，接近 $O(n^2)$

4.3.4 优缺点分析

优点：

- 无需预设簇数
- 能发现任意形状的簇
- 能识别噪声点
- 对输入顺序不敏感（确定性结果）

缺点：

- 参数敏感（ε, $MinPts$）
- 不适合密度差异大的数据
- 高维数据效果差（维度灾难）
- 大数据集计算量大

改进变体：

- **OPTICS**：处理密度不均匀数据
- **HDBSCAN**：自适应参数选择
- **DENCLUE**：基于密度函数的聚类

4.4 层次聚类详解

4.4.1 凝聚式层次聚类

Algorithm 4.2 AGNES凝聚层次聚类

输入: 数据集 X，距离度量，链接方法

输出: 树状图（Dendrogram）

初始化: 每个点为一个簇：$C_i = \{\mathbf{x}_i\}$

```
while 簇数 > 1 do
    计算所有簇对之间的距离矩阵
    找到最近的两个簇：(C_i, C_j) = arg min d(C_p, C_q)
    合并：C_new = C_i ∪ C_j
    记录合并信息和距离
    更新距离矩阵
end while
return 树状图
```

4.4.2 链接方法（Linkage Criteria）

1. **单链接（Single Linkage）：**

$$d(C_i, C_j) = \min_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} d(\mathbf{x}, \mathbf{y}) \tag{4.11}$$

   易产生"链式效应"，适合细长簇。

2. **全链接（Complete Linkage）：**

$$d(C_i, C_j) = \max_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} d(\mathbf{x}, \mathbf{y}) \tag{4.12}$$

   产生紧凑的球形簇。

3. **平均链接（Average Linkage）：**

$$d(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{\mathbf{x} \in C_i} \sum_{\mathbf{y} \in C_j} d(\mathbf{x}, \mathbf{y}) \tag{4.13}$$

   折中方案，常用。

4. **Ward法：**

$$d(C_i, C_j) = \frac{|C_i||C_j|}{|C_i| + |C_j|} \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|^2 \tag{4.14}$$

最小化合并后的方差增量，效果通常最好。

4.4.3 树状图切割

- **固定高度切割：** 在指定距离处切割
- **固定簇数切割：** 获得指定数量的簇
- **动态切割：** 根据不一致系数自动确定

4.5 高斯混合模型（GMM）

4.5.1 模型定义

GMM假设数据由多个高斯分布混合生成：

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k N(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k) \tag{4.15}$$

其中 π_k 为混合系数，$\sum \pi_k = 1$。

4.5.2 EM算法求解

Algorithm 4.3 GMM的EM算法

输入： 数据 X，簇数 K

输出： 模型参数 $\{\pi_k, \boldsymbol{\mu}_k, \Sigma_k\}$

初始化： 随机初始化参数

repeat

  E步： 计算责任度

  $\gamma(z_{nk}) = \frac{\pi_k N(\mathbf{x}_n|\boldsymbol{\mu}_k,\Sigma_k)}{\sum_j \pi_j N(\mathbf{x}_n|\boldsymbol{\mu}_j,\Sigma_j)}$

  M步： 更新参数

  $N_k = \sum_n \gamma(z_{nk})$

  $\boldsymbol{\mu}_k = \frac{1}{N_k}\sum_n \gamma(z_{nk})\mathbf{x}_n$

  $\Sigma_k = \frac{1}{N_k}\sum_n \gamma(z_{nk})(\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^T$

  $\pi_k = \frac{N_k}{N}$

until 对数似然收敛

return 模型参数和簇标签

优势： 软聚类（隶属度概率）、灵活协方差结构、概率框架。

4.6 谱聚类（Spectral Clustering）

4.6.1 算法原理

将聚类问题转化为图划分问题：

1. 构建相似图，计算邻接矩阵 W
2. 计算拉普拉斯矩阵 $L = D - W$（D 为度矩阵）
3. 计算 L 的前 K 个特征向量
4. 对特征向量矩阵的行进行K-Means聚类

适用场景：非凸形状簇、流形结构数据。

4.7 聚类算法选择指南

表 4.1: 聚类算法选择决策表

数据特征	簇形状	规模	推荐算法	备注
球形、等大小	凸形	大	K-Means	快速高效
任意形状	非凸	中小	DBSCAN	需调参
层次结构	任意	中小	层次聚类	可视化好
概率解释	椭球	中	GMM	软聚类
流形结构	非凸	中	谱聚类	计算复杂
高维稀疏	-	大	Mini-Batch KM	降维后更好
含噪声	任意	大	HDBSCAN	鲁棒性强

4.8 实际应用建议

1. **数据预处理：**
   - 标准化/归一化（尤其对距离敏感的算法）
   - 降维（PCA、t-SNE）后再聚类
   - 处理缺失值和异常值
2. **多次运行：**
   - K-Means等随机初始化算法应多次运行取最优

- 使用不同的随机种子

3. **结果验证：**

   - 使用多个评估指标
   - 可视化检查结果合理性
   - 领域专家验证

4. **业务解释：**

   - 分析各簇的特征画像
   - 给簇赋予业务含义
   - 制定针对性策略

Part V

有监督分类算法

Chapter 5

有监督分类算法

分类是有监督学习的核心任务，根据带标签的训练数据学习分类模型，对未知样本进行类别预测。

5.1 Logistic逻辑回归

5.1.1 数学模型

逻辑回归虽名为"回归"，实为二分类模型。使用Sigmoid函数将线性组合映射到概率：

$$P(y=1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}+b) = \frac{1}{1+e^{-(\mathbf{w}^T\mathbf{x}+b)}} \tag{5.1}$$

5.1.2 参数估计

通过最大化对数似然函数求解：

$$L(\mathbf{w},b) = \sum_{i=1}^{n}[y_i \log p_i + (1-y_i)\log(1-p_i)] \tag{5.2}$$

使用梯度下降或牛顿法优化。

5.1.3 多分类扩展

- **One-vs-Rest (OvR)**：训练K个二分类器

- **Softmax回归**：直接建模多类概率

$$P(y=k|\mathbf{x}) = \frac{e^{\mathbf{w}_k^T\mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T\mathbf{x}}} \tag{5.3}$$

5.2 支持向量机 (SVM)

5.2.1 最大间隔分类器

SVM寻找最优超平面，使两类样本的间隔最大化：

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \quad i = 1, \ldots, n \end{aligned} \tag{5.4}$$

5.2.2 软间隔与核技巧

软间隔：引入松弛变量 ξ_i 允许误分类：

$$\min \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \tag{5.5}$$

核技巧：将数据映射到高维空间处理非线性：

- 线性核：$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T\mathbf{x}_j$
- 多项式核：$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma\mathbf{x}_i^T\mathbf{x}_j + r)^d$
- RBF核：$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$
- Sigmoid核：$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma\mathbf{x}_i^T\mathbf{x}_j + r)$

5.3 决策树

5.3.1 ID3/C4.5算法

基于信息增益选择分裂属性：

$$\text{Information Gain} = H(D) - \sum_{v=1}^{V}\frac{|D^v|}{|D|}H(D^v) \tag{5.6}$$

其中 $H(D) = -\sum_{k=1}^{K} p_k \log_2 p_k$ 为熵。

C4.5使用信息增益率克服取值数目偏向：

$$\text{Gain Ratio} = \frac{\text{Gain}}{H_A(D)} \tag{5.7}$$

5.3.2 CART算法

使用基尼指数选择分裂点：

$$\text{Gini}(D) = 1 - \sum_{k=1}^{K} p_k^2 \tag{5.8}$$

5.3.3 剪枝策略

- **预剪枝**：在构建过程中提前停止
- **后剪枝**：先生成完整树，再自底向上剪枝

5.4 随机森林

5.4.1 算法原理

随机森林是Bagging集成方法，通过以下步骤构建：

1. Bootstrap采样生成多个训练子集
2. 在每个节点随机选择部分特征
3. 独立训练多棵决策树
4. 投票（分类）或平均（回归）得到最终预测

5.4.2 优缺点

优点：抗过拟合、能处理高维数据、可评估特征重要性；**缺点**：模型复杂、解释性差、对小数据集效果一般。

5.5 K近邻 (KNN)

5.5.1 算法描述

对于测试样本，找到训练集中最近的K个邻居，通过投票决定类别。

距离度量：

- 欧氏距离：$d = \sqrt{\sum (x_i - y_i)^2}$
- 曼哈顿距离：$d = \sum |x_i - y_i|$
- Minkowski距离：$d = (\sum |x_i - y_i|^p)^{1/p}$

5.5.2 优缺点

优点：简单直观、无需训练、天然支持多分类；**缺点**：预测慢、维度灾难、需要标准化、K值选择敏感。

5.6 朴素贝叶斯

5.6.1 贝叶斯定理

$$P(y|\mathbf{x}) = \frac{P(y)P(\mathbf{x}|y)}{P(\mathbf{x})} \tag{5.9}$$

5.6.2 条件独立性假设

朴素贝叶斯假设特征之间条件独立：

$$P(\mathbf{x}|y) = \prod_{j=1}^{d} P(x_j|y) \tag{5.10}$$

5.6.3 变体

- **高斯朴素贝叶斯**：连续特征服从正态分布
- **多项式朴素贝叶斯**：离散计数特征，适合文本分类
- **伯努利朴素贝叶斯**：二元特征

5.7 AdaBoost提升算法

5.7.1 算法原理

AdaBoost迭代训练弱分类器，每轮增加被错误分类样本的权重：

Algorithm 5.1 AdaBoost算法

输入: 训练数据 $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$，弱学习器数量 T

输出: 强分类器 $H(\mathbf{x})$

初始化: 初始化样本权重 $w_i = 1/n$

1: **for** $t = 1$ to T **do**
2: 使用权重 w_i 训练弱分类器 h_t
3: 计算加权误差：$\epsilon_t = \sum_{i:h_t(\mathbf{x}_i)\neq y_i} w_i$
4: 计算分类器权重：$\alpha_t = \frac{1}{2}\ln\frac{1-\epsilon_t}{\epsilon_t}$
5: 更新样本权重：$w_i \leftarrow w_i \exp(-\alpha_t y_i h_t(\mathbf{x}_i))$
6: 归一化权重
7: **end for**
8: 强分类器：$H(\mathbf{x}) = \text{sign}(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}))$
9: **return** $H(\mathbf{x})$

5.8 分类模型评估

- **混淆矩阵**：TP, TN, FP, FN
- **准确率**：$\frac{TP+TN}{TP+TN+FP+FN}$
- **精确率**：$\frac{TP}{TP+FP}$
- **召回率**：$\frac{TP}{TP+FN}$
- **F1分数**：$2 \cdot \frac{\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}$
- **ROC曲线与AUC**：评估不同阈值下的性能
- **交叉验证**：K折交叉验证评估泛化能力

Part VI

图论算法

Chapter 6

图论算法

图论在数学建模中广泛应用，包括网络优化、路径规划、资源分配等问题。本章介绍经典的图论算法。

6.1 最短路径算法

6.1.1 Dijkstra算法

Dijkstra算法求解单源最短路径，适用于非负权图。

Algorithm 6.1 Dijkstra最短路径算法

输入: 图 $G = (V, E)$，源点 s，边权 $w(u, v) \geq 0$

输出: 从 s 到所有顶点的最短距离 $dist[]$

初始化:

对所有 $v \in V$：$dist[v] \leftarrow \infty$
$dist[s] \leftarrow 0$
$S \leftarrow \emptyset$ （已确定最短路径的顶点集合）
使用优先队列 Q，将所有顶点加入
while Q 非空 **do**
    $u \leftarrow$ 从 Q 中提取 $dist$ 最小的顶点
    $S \leftarrow S \cup \{u\}$
    for 每个邻居 v of u **do**
        if $dist[u] + w(u, v) < dist[v]$ **then**
            $dist[v] \leftarrow dist[u] + w(u, v)$
            更新 Q 中 v 的优先级
            $prev[v] \leftarrow u$ （记录前驱）
        end if
    end for
end while
return $dist[]$ 和前驱数组 $prev[]$

时间复杂度： $O((V + E) \log V)$ 使用二叉堆。

注意： 不能处理负权边。

6.1.2 Bellman-Ford算法

可处理含负权边的图，并能检测负环。

Algorithm 6.2 Bellman-Ford算法

输入: 图 $G = (V, E)$，源点 s

输出: 最短距离或检测到负环

初始化:

对所有 $v \in V$：$dist[v] \leftarrow \infty$
$dist[s] \leftarrow 0$
for $i = 1$ to $|V| - 1$ **do**
    for 每条边 $(u, v) \in E$ **do**
        if $dist[u] + w(u, v) < dist[v]$ **then**
            $dist[v] \leftarrow dist[u] + w(u, v)$
        end if
    end for
end for
检测负环:
for 每条边 $(u, v) \in E$ **do**
    if $dist[u] + w(u, v) < dist[v]$ **then**
        return “存在负环”
    end if
end for
return $dist[]$

时间复杂度： $O(VE)$。

6.1.3 Floyd-Warshall算法

求解所有顶点对之间的最短路径。

Algorithm 6.3 Floyd-Warshall全源最短路径

输入: 图的邻接矩阵 W，$W[i][j]$ 为边 (i,j) 的权重
输出: 最短距离矩阵 D
初始化: $D \leftarrow W$

```
for k = 1 to n do
    for i = 1 to n do
        for j = 1 to n do
            D[i][j] ← min(D[i][j], D[i][k] + D[k][j])
        end for
    end for
end for
return D
```

时间复杂度： $O(V^3)$，适合小规模稠密图。

6.1.4 SPFA算法

Bellman-Ford的队列优化版本，平均性能更好。

6.2 最小生成树

6.2.1 Kruskal算法

基于贪心策略，从小到大选择边，避免形成环。

Algorithm 6.4 Kruskal最小生成树算法

输入: 无向图 $G = (V, E)$，边权 $w(e)$

输出: 最小生成树的边集合 T

初始化:

```
将所有边按权重升序排序
T ← ∅
初始化并查集，每个顶点独立为一个集合
for 每条边 (u, v) （按权重从小到大） do
    if u 和 v 不在同一集合中 then
        T ← T ∪ {(u, v)}
        合并 u 和 v 所在集合
    end if
    if |T| = |V| − 1 then

    end if
end for
return T
```

时间复杂度： $O(E \log E)$。

6.2.2 Prim算法

从任意顶点开始，逐步扩展生成树。

Algorithm 6.5 Prim最小生成树算法

输入: 无向连通图 $G = (V, E)$，边权 $w(e)$

输出: 最小生成树

初始化:

选择任意起始顶点 r
对所有 v：$key[v] \leftarrow \infty$
$key[r] \leftarrow 0$
$prev[r] \leftarrow$ NULL
优先队列 Q 包含所有顶点，按 key 值排序
while Q 非空 **do**
    $u \leftarrow$ 从 Q 中提取 key 最小的顶点
    for 每个邻居 v of u **do**
        if $v \in Q$ 且 $w(u, v) < key[v]$ **then**
            $key[v] \leftarrow w(u, v)$
            $prev[v] \leftarrow u$
            更新 Q 中 v 的优先级
        end if
    end for
end while
return $\{(prev[v], v) : v \neq r\}$

时间复杂度： $O((V + E) \log V)$。

选择建议： 稀疏图用Kruskal，稠密图用Prim。

6.3 网络流算法

6.3.1 最大流问题

定义 6.1. 给定有向图 $G = (V, E)$，源点 s，汇点 t，每条边有容量 $c(u, v)$，求从 s 到 t 的最大流量。

6.3.2 Ford-Fulkerson方法

基于增广路思想：

Algorithm 6.6 Ford-Fulkerson最大流算法

输入: 网络 $G = (V, E)$，源点 s，汇点 t，容量 $c(u, v)$

输出: 最大流值

初始化:

对所有边 (u, v): $f(u, v) \leftarrow 0$
while 存在从 s 到 t 的增广路 p **do**
    $c_f(p) \leftarrow \min\{c_f(u, v) : (u, v) \in p\}$ （残差容量）
    for 每条边 $(u, v) \in p$ **do**
        $f(u, v) \leftarrow f(u, v) + c_f(p)$
        $f(v, u) \leftarrow f(v, u) - c_f(p)$
    end for
end while
return $\sum_v f(s, v)$

6.3.3 Dinic算法

更高效的最大流算法，使用分层图和阻塞流。

核心步骤:

1. BFS构建分层图
2. DFS寻找阻塞流
3. 重复直到无法增广

时间复杂度: $O(V^2E)$，实际性能通常更好。

6.3.4 最小费用最大流

在最大流基础上，考虑边的单位流量费用，求费用最小的最大流。可使用SPFA或Dijkstra（势能法）寻找最短增广路。

6.4 二分图匹配

6.4.1 匈牙利算法

求解二分图最大匹配。

Algorithm 6.7 匈牙利算法

输入: 二分图 $G = (X \cup Y, E)$

输出: 最大匹配

初始化: 所有顶点未匹配

```
for 每个顶点 x ∈ X do
    初始化访问标记
    if AUGMENT(x) then
        匹配数加1
    end if
end for
return 最大匹配
function AUGMENT(u)
    for 每个与 u 相邻的 v do
        if v 未被访问 then
            标记 v 已访问
            if v 未匹配或 AUGMENT(match[v]) then
                match[v] ← u
                return True
            end if
        end if
    end for
    return False
end function
```

时间复杂度： $O(VE)$。

6.4.2 Kuhn-Munkres算法 (KM算法)

求解二分图最优匹配（带权最大匹配）。

6.5 关键路径法 (CPM)

用于项目调度，确定关键工序和最短工期。

关键概念：

- **最早开始时间 (ES)：**活动可以开始的最早时间
- **最晚开始时间 (LS)：**不延误项目的最晚开始时间
- **时差 (Slack)：**$LS - ES$，时差为0的活动在关键路径上

6.6 图论算法应用

- **交通网络**：最短路径、最大流
- **任务调度**：拓扑排序、关键路径
- **资源分配**：匹配、网络流
- **社交网络**：中心性分析、社区发现
- **电路设计**：最小生成树、布线

Part VII

插值与曲线拟合

Chapter 7

插值与曲线拟合

插值和拟合是通过离散数据点构造连续函数的重要方法，广泛应用于数据补全、平滑处理和趋势分析。

7.1 拉格朗日插值

7.1.1 基本原理

给定 $n+1$ 个节点 $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$，构造 n 次多项式 $L_n(x)$ 使得 $L_n(x_i) = y_i$。

基函数：

$$l_i(x) = \prod_{j=0, j\neq i}^{n} \frac{x - x_j}{x_i - x_j} = \frac{(x-x_0)\cdots(x-x_{i-1})(x-x_{i+1})\cdots(x-x_n)}{(x_i-x_0)\cdots(x_i-x_{i-1})(x_i-x_{i+1})\cdots(x_i-x_n)} \tag{7.1}$$

插值多项式：

$$L_n(x) = \sum_{i=0}^{n} y_i l_i(x) \tag{7.2}$$

7.1.2 性质

- **存在唯一性：** 对于给定的 $n+1$ 个点，存在唯一的次数不超过 n 的插值多项式
- **基函数性质：** $l_i(x_j) = \delta_{ij}$（Kronecker delta）
- **误差公式：**

$$f(x) - L_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i) \tag{7.3}$$

其中 ξ 在数据点范围内。

7.1.3 优缺点

优点：形式简洁、理论完善；**缺点：**高次插值易产生龙格现象（Runge phenomenon），即在区间端点附近剧烈振荡。

7.2 牛顿插值

7.2.1 差商定义

定义 7.1 (差商).

$$\begin{aligned}
&\text{零阶差商：} && f[x_i] = f(x_i) \\
&\text{一阶差商：} && f[x_i, x_j] = \frac{f(x_j) - f(x_i)}{x_j - x_i} \\
&k\text{阶差商：} && f[x_0, \ldots, x_k] = \frac{f[x_1, \ldots, x_k] - f[x_0, \ldots, x_{k-1}]}{x_k - x_0}
\end{aligned}$$

性质： 差商具有对称性，与节点顺序无关。

7.2.2 牛顿插值多项式

$$N_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots + f[x_0, \ldots, x_n] \prod_{i=0}^{n-1} (x - x_i) \tag{7.4}$$

7.2.3 算法优势

- **增量更新：** 增加新节点时，只需计算新的差商，无需重新计算全部
- **数值稳定性：** 相比拉格朗日插值更稳定
- **计算效率：** 使用差商表，时间复杂度 $O(n^2)$

Algorithm 7.1 牛顿插值差商表构建

输入: 节点 $(x_i, y_i), i = 0, \ldots, n$

输出: 差商表

初始化：$f[x_i] = y_i$ for $i = 0, \ldots, n$
for $j = 1$ to n **do**
    for $i = n$ down to j **do**
        $f[x_{i-j}, \ldots, x_i] = \frac{f[x_{i-j+1}, \ldots, x_i] - f[x_{i-j}, \ldots, x_{i-1}]}{x_i - x_{i-j}}$
    end for
end for
return 对角线元素即为所需差商

7.3 分段线性插值

7.3.1 基本原理

将区间 $[a,b]$ 划分为子区间，在每个子区间上使用线性插值：

$$S(x) = \frac{x_{i+1} - x}{x_{i+1} - x_i} y_i + \frac{x - x_i}{x_{i+1} - x_i} y_{i+1}, \quad x \in [x_i, x_{i+1}] \tag{7.5}$$

优点：简单、稳定、无振荡；**缺点**：在节点处不可导，光滑性差。

7.4 三次样条插值

7.4.1 定义

定义 7.2 (三次样条). *函数 $S(x)$ 称为三次样条，如果：*

1. *在每个子区间 $[x_i, x_{i+1}]$ 上是三次多项式*
2. *$S(x_i) = y_i$（插值条件）*
3. *$S(x)$ 在 $[a,b]$ 上二阶导数连续*

7.4.2 边界条件

需要两个额外条件确定唯一解：

- **自然样条**： $S''(x_0) = S''(x_n) = 0$
- **固定边界**： $S'(x_0) = f'_0, S'(x_n) = f'_n$
- **非扭结**： $S'''(x)$ 在端点连续

7.4.3 三弯矩方程

设 $M_i = S''(x_i)$，则：

$$\mu_i M_{i-1} + 2M_i + \lambda_i M_{i+1} = d_i, \quad i = 1, \ldots, n-1 \tag{7.6}$$

其中：

$$
\begin{aligned}
h_i &= x_{i+1} - x_i \\
\mu_i &= \frac{h_{i-1}}{h_{i-1}+h_i}, \quad \lambda_i = \frac{h_i}{h_{i-1}+h_i} \\
d_i &= \frac{6}{h_{i-1}+h_i}\left(\frac{y_{i+1}-y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right)
\end{aligned}
$$

这是三对角线性方程组，可用追赶法高效求解。

7.4.4 优势

- 全局光滑（二阶导数连续）
- 避免高次多项式的龙格现象
- 局部性好，修改一个节点只影响邻近区域
- 广泛应用於CAD、图形学、工程建模

7.5 埃尔米特插值

7.5.1 问题描述

不仅要求函数值匹配，还要求导数值匹配：

$$
H(x_i) = y_i, \quad H'(x_i) = y'_i, \quad i = 0, \ldots, n \tag{7.7}
$$

7.5.2 构造方法

对于两点 (x_0, y_0, y'_0) 和 (x_1, y_1, y'_1)，三次Hermite插值为：

$$
H_3(x) = y_0 h_{00}(t) + y_1 h_{01}(t) + y'_0 h_{10}(t) + y'_1 h_{11}(t) \tag{7.8}
$$

其中 $t = \frac{x-x_0}{x_1-x_0}$，$h_{ij}$ 为Hermite基函数。

7.6 最小二乘拟合

7.6.1 基本原理

与插值不同，拟合不要求曲线通过所有数据点，而是寻找最优近似。

目标： 最小化残差平方和

$$\min_{\boldsymbol{\beta}} J(\boldsymbol{\beta}) = \sum_{i=1}^{m} [y_i - f(\mathbf{x}_i; \boldsymbol{\beta})]^2 \tag{7.9}$$

7.6.2 线性最小二乘

对于线性模型 $y = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$：

矩阵形式：$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$

正规方程：

$$\mathbf{X}^T\mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}^T\mathbf{y} \tag{7.10}$$

解析解：

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{7.11}$$

7.6.3 非线性最小二乘

对于非线性模型，使用迭代方法：

Algorithm 7.2 Gauss-Newton算法

输入: 初始参数 $\boldsymbol{\beta}_0$，数据 $(\mathbf{x}_i, y_i)$

输出: 最优参数 $\hat{\boldsymbol{\beta}}$

1: $k \leftarrow 0$
2: **while** 未收敛 **do**
3: 计算残差：$r_i = y_i - f(\mathbf{x}_i; \boldsymbol{\beta}_k)$
4: 计算Jacobian矩阵：$J_{ij} = \frac{\partial f(\mathbf{x}_i; \boldsymbol{\beta}_k)}{\partial \beta_j}$
5: 求解：$(\mathbf{J}^T\mathbf{J})\boldsymbol{\delta} = \mathbf{J}^T\mathbf{r}$
6: 更新：$\boldsymbol{\beta}_{k+1} = \boldsymbol{\beta}_k + \boldsymbol{\delta}$
7: $k \leftarrow k + 1$
8: **end while**
9: **return** $\hat{\boldsymbol{\beta}} = \boldsymbol{\beta}_k$

7.6.4 常用拟合模型

表 7.1: 常见拟合函数

类型	模型	适用场景
线性	$y = a + bx$	直线趋势
多项式	$y = a_0 + a_1 x + \cdots + a_n x^n$	一般曲线
指数	$y = ae^{bx}$	增长/衰减
对数	$y = a + b \ln x$	饱和增长
幂函数	$y = ax^b$	比例关系
Logistic	$y = \frac{L}{1+e^{-k(x-x_0)}}$	S型增长

7.7 方法选择指南

表 7.2: 插值与拟合方法选择

需求	数据特点	推荐方法	备注
精确通过所有点	少量、精确	拉格朗日/牛顿	$n < 10$
光滑曲线	中等数量	三次样条	最常用
快速简单	大量数据	分段线性	实时应用
含噪声数据	有测量误差	最小二乘拟合	不强制通过
导数已知	Hermite数据	Hermite插值	特殊场景
全局趋势	大样本	多项式拟合	低阶为宜

7.8 注意事项

1. **避免高次插值：** 次数 > 10 时容易出现龙格现象，改用分段或样条
2. **节点分布：** 切比雪夫节点可减小最大误差
3. **过拟合风险：** 拟合时多项式次数不宜过高，使用交叉验证选择
4. **数值稳定性：** 大规模问题时使用QR分解或SVD求解正规方程
5. **外推谨慎：** 插值和拟合都只在内插区间可靠，外推需特别小心

Part VIII

综合评价与多准则决策

Chapter 8

综合评价与多准则决策

综合评价是将多个指标转化为一个综合指标，对方案进行排序或分级的方法。广泛应用于项目评估、绩效考评、方案优选等场景。

8.1 AHP层次分析法详解

8.1.1 基本原理

AHP通过分层结构将复杂决策问题分解为目标、准则、子准则和方案层，通过两两比较确定权重。

层次结构：

- 目标层：最终决策目标
- 准则层：评价维度（如成本、质量、时间）
- 子准则层：更细的评价指标
- 方案层：待选方案

8.1.2 判断矩阵构建

采用1-9标度法：

表 8.1: Saaty 1-9标度

标度	含义
1	两个因素同样重要
3	前者比后者稍重要
5	前者比后者明显重要
7	前者比后者强烈重要
9	前者比后者极端重要
2,4,6,8	相邻判断的中间值
倒数	若 $a_{ij} = k$，则 $a_{ji} = 1/k$

判断矩阵：

$$A = \begin{bmatrix} 1 & a_{12} & \cdots & a_{1n} \\ 1/a_{12} & 1 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 1/a_{1n} & 1/a_{2n} & \cdots & 1 \end{bmatrix} \tag{8.1}$$

8.1.3 权重计算

1. 特征向量法（精确解）：

求解 $A\mathbf{w} = \lambda_{max}\mathbf{w}$，归一化后的特征向量即为权重。

2. 算术平均法（近似解）：

$$w_i = \frac{1}{n}\sum_{j=1}^{n}\frac{a_{ij}}{\sum_{k=1}^{n} a_{kj}} \tag{8.2}$$

3. 几何平均法（推荐）：

$$w_i = \frac{\left(\prod_{j=1}^{n} a_{ij}\right)^{1/n}}{\sum_{i=1}^{n}\left(\prod_{j=1}^{n} a_{ij}\right)^{1/n}} \tag{8.3}$$

Algorithm 8.1 AHP完整计算流程

输入: 判断矩阵 $A_{n\times n}$

输出: 权重向量 $\mathbf{w}$ 及一致性检验结果

Step 1: 计算权重

计算每列之和：$S_j = \sum_{i=1}^{n} a_{ij}$

归一化矩阵：$\bar{a}_{ij} = a_{ij}/S_j$

计算权重：$w_i = \frac{1}{n}\sum_{j=1}^{n} \bar{a}_{ij}$

Step 2: 计算最大特征值

$$Aw = \begin{bmatrix} \sum_j a_{1j}w_j \\ \vdots \\ \sum_j a_{nj}w_j \end{bmatrix}$$

$\lambda_{max} = \frac{1}{n}\sum_{i=1}^{n} \frac{(Aw)_i}{w_i}$

Step 3: 一致性检验

$CI = \frac{\lambda_{max}-n}{n-1}$

查表获取 RI 值

$CR = \frac{CI}{RI}$

if $CR < 0.1$ **then**

    通过一致性检验

else

    需要调整判断矩阵

end if

return $\mathbf{w}, \lambda_{max}, CR$

8.1.4 随机一致性指标 RI

表 8.2: RI值表

n	1	2	3	4	5	6	7	8	9	10
RI	0	0	0.58	0.90	1.12	1.24	1.32	1.41	1.45	1.49

8.1.5 层次总排序

若准则层权重为 $\mathbf{w}^{(1)}$，方案层对各准则的权重矩阵为 $W^{(2)}$，则方案的综合权重为：

$$\mathbf{w}_{total} = W^{(2)}\mathbf{w}^{(1)} \tag{8.4}$$

8.1.6 优缺点分析

优点：

- 系统性强，逻辑清晰
- 适用于难以量化的定性指标
- 可检验判断的一致性
- 广泛应用，易于理解

缺点：

- 主观性强，依赖专家经验
- 指标过多时判断矩阵规模大
- 可能存在秩逆转问题
- 1-9标度的合理性有争议

8.2 熵权法 (Entropy Weight Method)

8.2.1 基本原理

熵权法基于信息论，根据指标的离散程度客观确定权重。指标变异越大，提供的信息越多，权重越高。

8.2.2 计算步骤

Algorithm 8.2 熵权法完整流程

输入: 决策矩阵 $X_{m\times n}$（m个方案，n个指标）

输出: 指标权重 **w** 及各方案综合得分

Step 1: 数据标准化

for 每个指标 j **do**

  if 正向指标（越大越好） **then**

    $p_{ij} = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)}$

  else负向指标（越小越好）

    $p_{ij} = \frac{\max(x_j) - x_{ij}}{\max(x_j) - \min(x_j)}$

  end if

end for

Step 2: 归一化

$P_{ij} = \frac{p_{ij}}{\sum_{i=1}^{m} p_{ij}}$ （确保 $P_{ij} > 0$）

Step 3: 计算熵值

for 每个指标 j **do**

  $e_j = -\frac{1}{\ln m} \sum_{i=1}^{m} P_{ij} \ln P_{ij}$

end for

Step 4: 计算差异系数

$d_j = 1 - e_j$

Step 5: 确定权重

$w_j = \frac{d_j}{\sum_{k=1}^{n} d_k}$

Step 6: 计算综合得分

$S_i = \sum_{j=1}^{n} w_j p_{ij}$

return 权重 **w** 和得分 **S**

8.2.3 注意事项

- 若 $P_{ij} = 0$，定义 $P_{ij} \ln P_{ij} = 0$
- 熵权法完全依赖数据，可能忽视实际重要性
- 建议与主观赋权法结合使用

8.3 TOPSIS法详解

8.3.1 基本原理

TOPSIS（Technique for Order Preference by Similarity to Ideal Solution）通过计算各方案与理想解和负理想解的距离来排序。

核心思想：最优方案应距离正理想解最近，距离负理想解最远。

8.3.2 计算步骤

Algorithm 8.3 TOPSIS完整算法

输入: 决策矩阵 $X_{m\times n}$，权重向量 $\mathbf{w}$

输出: 方案排序及相对接近度

Step 1: 向量归一化

$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{i=1}^{m} x_{ij}^2}}$

Step 2: 加权归一化

$v_{ij} = w_j \cdot r_{ij}$

Step 3: 确定理想解

正理想解：$v_j^+ = \max_i(v_{ij})$（正向指标）或 $\min_i(v_{ij})$（负向指标）

负理想解：$v_j^- = \min_i(v_{ij})$（正向指标）或 $\max_i(v_{ij})$（负向指标）

Step 4: 计算距离

$D_i^+ = \sqrt{\sum_{j=1}^{n}(v_{ij} - v_j^+)^2}$

$D_i^- = \sqrt{\sum_{j=1}^{n}(v_{ij} - v_j^-)^2}$

Step 5: 计算相对接近度

$C_i = \frac{D_i^-}{D_i^+ + D_i^-}$

Step 6: 排序

按 C_i 从大到小排序（$C_i \in [0,1]$，越接近1越优）

return 排序结果和 C_i 值

8.3.3 示例

某公司选择供应商，3个候选，4个指标（价格、质量、交货期、服务）：

方案 C	价格	质量	交货期	服务	D^+	D^-
A	0.30	0.85	5	7	0.15	0.28
0.65						
B	0.35	0.90	4	8	0.12	0.32
0.73						
C	0.28	0.80	6	6	0.22	0.18
0.45						

表 8.3: TOPSIS评价结果

排序：B ¿ A ¿ C

8.4 灰色关联分析 (GRA)

8.4.1 基本原理

灰色关联分析通过计算序列曲线的几何相似程度来判断联系紧密程度，适合小样本。

8.4.2 计算步骤

1. **确定参考序列** X_0（理想方案）和比较序列 X_i

2. **无量纲化处理：**

$$x_i'(k) = \frac{x_i(k)}{x_i(1)} \quad \text{或} \quad x_i'(k) = \frac{x_i(k) - \min x_i}{\max x_i - \min x_i} \tag{8.5}$$

3. **计算关联系数：**

$$\xi_i(k) = \frac{\min\min |x_0(k) - x_i(k)| + \rho \max\max |x_0(k) - x_i(k)|}{|x_0(k) - x_i(k)| + \rho \max\max |x_0(k) - x_i(k)|} \tag{8.6}$$

其中 $\rho \in [0, 1]$ 为分辨系数，通常取0.5。

4. **计算关联度：**

$$r_i = \frac{1}{n} \sum_{k=1}^{n} \xi_i(k) \tag{8.7}$$

5. **排序：**按 r_i 从大到小排序

8.5 模糊综合评价 (FCE)

8.5.1 基本概念

模糊综合评价利用模糊数学的隶属度理论，将定性评价转化为定量评价。

8.5.2 实施步骤

Algorithm 8.4 模糊综合评价流程

输入: 因素集 $U = \{u_1, \ldots, u_m\}$，评语集 $V = \{v_1, \ldots, v_n\}$

输出: 综合评价结果

Step 1: 确定权重向量 $\mathbf{w} = (w_1, \ldots, w_m)$

（可用AHP、熵权法等确定）

Step 2: 构建隶属度矩阵 R

r_{ij} 表示因素 u_i 对评语 v_j 的隶属度

$$R = \begin{bmatrix} r_{11} & \cdots & r_{1n} \\ \vdots & \ddots & \vdots \\ r_{m1} & \cdots & r_{mn} \end{bmatrix}$$

Step 3: 模糊合成

$B = \mathbf{w} \circ R = (b_1, \ldots, b_n)$

常用算子：$M(\wedge, \vee)$ 或 $M(\cdot, +)$

Step 4: 去模糊化

最大隶属度原则：$v^* = \arg\max_j b_j$

或加权平均：$S = \sum_{j=1}^{n} b_j \cdot score(v_j)$

return 评价等级和得分

8.5.3 隶属函数设计

三角形隶属函数：

$$\mu(x) = \begin{cases} 0 & x \le a \\ \frac{x-a}{b-a} & a < x \le b \\ \frac{c-x}{c-b} & b < x \le c \\ 0 & x > c \end{cases} \tag{8.8}$$

8.6 DEA数据包络分析

8.6.1 基本原理

DEA用于评估多投入多产出系统的相对效率，无需预设权重。

8.6.2 CCR模型（规模报酬不变）

对于 n 个决策单元(DMU)，每个有 m 种投入和 s 种产出：

$$\begin{aligned} \max_{\mathbf{u},\mathbf{v}} \quad & \theta = \frac{\sum_{r=1}^{s} u_r y_{rk}}{\sum_{i=1}^{m} v_i x_{ik}} \\ \text{s.t.} \quad & \frac{\sum_{r=1}^{s} u_r y_{rj}}{\sum_{i=1}^{m} v_i x_{ij}} \leq 1, \quad j = 1, \ldots, n \\ & u_r, v_i \geq 0 \end{aligned} \tag{8.9}$$

转化为线性规划：

$$\begin{aligned} \max \quad & \sum_{r=1}^{s} u_r y_{rk} \\ \text{s.t.} \quad & \sum_{i=1}^{m} v_i x_{ik} = 1 \\ & \sum_{r=1}^{s} u_r y_{rj} - \sum_{i=1}^{m} v_i x_{ij} \leq 0, \quad \forall j \\ & u_r, v_i \geq 0 \end{aligned} \tag{8.10}$$

8.6.3 效率解读

- $\theta = 1$：DEA有效（技术有效）
- $\theta < 1$：DEA无效，存在改进空间
- 可计算投入冗余和产出不足

8.7 VIKOR方法

8.7.1 基本原理

VIKOR兼顾群体效用最大化和个体遗憾最小化，提供折中解。

8.7.2 计算步骤

1. 确定最优值 f_j^* 和最差值 f_j^-

2. 计算 S_i 和 R_i：

$$S_i = \sum_{j=1}^{n} w_j \frac{f_j^* - f_{ij}}{f_j^* - f_j^-} \tag{8.11}$$

$$R_i = \max_j \left[w_j \frac{f_j^* - f_{ij}}{f_j^* - f_j^-} \right] \tag{8.12}$$

3. 计算 Q_i：

$$Q_i = v\frac{S_i - S^*}{S^- - S^*} + (1-v)\frac{R_i - R^*}{R^- - R^*} \tag{8.13}$$

   其中 v 通常为0.5。

4. 按 S, R, Q 分别排序

5. 确定妥协解（需满足可接受优势和稳定性条件）

8.8 方法选择指南

表 8.4: 综合评价方法选择

场景	推荐方法	数据需求	特点
主观判断为主	AHP	专家打分	系统性强
客观数据充足	熵权法	历史数据	客观公正
方案对比优选	TOPSIS	决策矩阵	直观合理
小样本贫信息	灰色关联	少量数据	适应性强
定性指标多	模糊综合	隶属度	处理模糊性
多投入多产出	DEA	面板数据	无需权重
权衡妥协解	VIKOR	决策矩阵	平衡优劣

8.9 组合评价方法

为提高评价结果的可靠性，可采用多种方法的组合：

- **权重组合**：AHP+熵权法的主客观组合权重
- **结果组合**：Borda计数法整合多种方法的排序
- **过程组合**：不同阶段使用不同方法

Part IX

随机仿真与概率模型

Chapter 9

随机仿真与概率模型

随机仿真和概率模型是处理不确定性问题的重要工具，广泛应用于风险评估、金融建模、可靠性分析等领域。

9.1 蒙特卡洛方法 (Monte Carlo Method)

9.1.1 基本原理

蒙特卡洛方法通过随机抽样来估计数值结果，核心思想是大数定律：当样本量足够大时，样本均值收敛到期望值。

基本公式：

$$E[g(X)] = \int g(x)f(x)dx \approx \frac{1}{N}\sum_{i=1}^{N} g(X_i) \tag{9.1}$$

其中 X_i 是从分布 $f(x)$ 中抽取的独立样本。

9.1.2 数值积分示例

计算定积分 $I = \int_a^b f(x)dx$：

Algorithm 9.1 蒙特卡洛数值积分

输入: 被积函数 $f(x)$，积分区间 $[a, b]$，样本数 N

输出: 积分估计值 $\hat{I}$

1: $\text{sum} \leftarrow 0$
2: **for** $i = 1$ to N **do**
3: 生成均匀随机数 $U \sim U(0, 1)$
4: $x_i \leftarrow a + (b - a)U$
5: $\text{sum} \leftarrow \text{sum} + f(x_i)$
6: **end for**
7: $\hat{I} \leftarrow \frac{b-a}{N} \cdot \text{sum}$
8: **return** $\hat{I}$

误差估计：

$$\text{Std Error} = \frac{(b-a)\sigma_f}{\sqrt{N}}, \quad \sigma_f^2 = \text{Var}[f(X)] \tag{9.2}$$

9.1.3 圆周率 π 估算

经典案例：在单位正方形内随机投点，统计落在内切圆内的比例。

$$\pi \approx 4 \times \frac{\text{圆内点数}}{\text{总点数}} \tag{9.3}$$

9.1.4 方差缩减技术

1. **对偶变量法（Antithetic Variates）：**

$$\hat{\mu} = \frac{1}{2N} \sum_{i=1}^{N} [g(U_i) + g(1 - U_i)] \tag{9.4}$$

2. **控制变量法（Control Variates）：**

$$\hat{Y} = \bar{Y} - \beta(\bar{X} - \mu_X) \tag{9.5}$$

3. **重要抽样（Importance Sampling）：**

$$E_f[g(X)] = E_h \left[g(X) \frac{f(X)}{h(X)} \right] \tag{9.6}$$

选择更接近被积函数的提议分布 $h(x)$。

4. **分层抽样（Stratified Sampling）：** 将样本空间划分为若干层，每层独立抽样。

9.2 马尔可夫链 (Markov Chain)

9.2.1 基本定义

定义 9.1 (马尔可夫性质). 随机过程 $\{X_t, t = 0, 1, 2, \ldots\}$ 具有马尔可夫性质，如果：

$$P(X_{t+1} = j | X_t = i, X_{t-1}, \ldots, X_0) = P(X_{t+1} = j | X_t = i) = p_{ij} \tag{9.7}$$

即未来状态仅依赖于当前状态，与历史无关。

9.2.2 转移矩阵

对于有限状态空间 $S = \{1, 2, \ldots, n\}$，转移矩阵为：

$$P = \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1n} \\ p_{21} & p_{22} & \cdots & p_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{n1} & p_{n2} & \cdots & p_{nn} \end{bmatrix} \tag{9.8}$$

性质：

- $p_{ij} \geq 0$
- $\sum_j p_{ij} = 1$（行和为1）
- P^n 的元素 $p_{ij}^{(n)}$ 表示 n 步转移概率

9.2.3 稳态分布

定理 9.1 (稳态分布存在性). *对于不可约、非周期的马尔可夫链，存在唯一的稳态分布 $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_n)$ 满足：*

$$\boldsymbol{\pi} P = \boldsymbol{\pi}, \quad \sum_i \pi_i = 1 \tag{9.9}$$

Algorithm 9.2 马尔可夫链稳态分布计算

输入： 转移矩阵 $P_{n \times n}$

输出： 稳态分布 $\boldsymbol{\pi}$

方法1：特征向量法

求解 $(P^T - I)\boldsymbol{\pi} = \mathbf{0}$，附加约束 $\sum \pi_i = 1$

方法2：幂迭代法

初始化 $\boldsymbol{\pi}^{(0)} = (1/n, \ldots, 1/n)$

repeat

  $\boldsymbol{\pi}^{(k+1)} \leftarrow \boldsymbol{\pi}^{(k)} P$

  归一化：$\boldsymbol{\pi}^{(k+1)} \leftarrow \boldsymbol{\pi}^{(k+1)} / \|\boldsymbol{\pi}^{(k+1)}\|_1$

until $\|\boldsymbol{\pi}^{(k+1)} - \boldsymbol{\pi}^{(k)}\| < \epsilon$

return $\boldsymbol{\pi}$

9.2.4 应用实例

1. 市场占有率预测

假设三个品牌A、B、C，月转移概率：

$$P = \begin{bmatrix} 0.7 & 0.2 & 0.1 \\ 0.1 & 0.8 & 0.1 \\ 0.1 & 0.1 & 0.8 \end{bmatrix} \tag{9.10}$$

初始份额 $\boldsymbol{\pi}^{(0)} = (0.4, 0.35, 0.25)$，长期稳态份额可通过迭代或求解线性方程组得到。

2. PageRank算法

Google的网页排名基于马尔可夫链的稳态分布：

$$\mathbf{r} = (1-\alpha)\frac{\mathbf{1}}{N} + \alpha M\mathbf{r} \tag{9.11}$$

其中 M 为链接转移矩阵，$\alpha \approx 0.85$ 为阻尼因子。

9.3 排队论 (Queueing Theory)

9.3.1 Kendall记号

排队系统用 $A/B/c/K/m/Z$ 表示：

- A：到达过程分布（M=Poisson, D=确定, G=一般）
- B：服务时间分布
- c：服务台数量
- K：系统容量（默认∞）
- m：顾客源数量（默认∞）
- Z：服务规则（FCFS, LCFS, Priority等）

9.3.2 M/M/1模型详解

假设：

- 到达过程：Poisson过程，速率 λ
- 服务时间：指数分布，速率 μ
- 单服务台，无限容量，FCFS

关键参数：

$$\rho = \frac{\lambda}{\mu} \quad \text{(利用率，需 } \rho < 1\text{)} \tag{9.12}$$

稳态概率：

$$P_n = (1-\rho)\rho^n, \quad n = 0, 1, 2, \ldots \tag{9.13}$$

性能指标：

$$L = \frac{\rho}{1-\rho} \quad \text{(系统中平均顾客数)} \tag{9.14}$$

$$L_q = \frac{\rho^2}{1-\rho} \quad \text{(队列中平均顾客数)} \tag{9.15}$$

$$W = \frac{1}{\mu-\lambda} \quad \text{(系统中平均停留时间)} \tag{9.16}$$

$$W_q = \frac{\rho}{\mu-\lambda} \quad \text{(队列中平均等待时间)} \tag{9.17}$$

Little公式： $L = \lambda W$, $L_q = \lambda W_q$

9.3.3 M/M/c模型

c 个并行服务台：

稳态概率：

$$P_0 = \left[\sum_{n=0}^{c-1} \frac{(\lambda/\mu)^n}{n!} + \frac{(\lambda/\mu)^c}{c!(1-\rho)}\right]^{-1} \tag{9.18}$$

其中 $\rho = \lambda/(c\mu)$。

平均队列长度：

$$L_q = \frac{P_0(\lambda/\mu)^c \rho}{c!(1-\rho)^2} \tag{9.19}$$

9.3.4 M/G/1模型

服务时间为一般分布：

Pollaczek-Khinchine公式：

$$L_q = \frac{\lambda^2 E[S^2]}{2(1-\rho)} = \frac{\rho^2(1+C_s^2)}{2(1-\rho)} \tag{9.20}$$

其中 $C_s^2 = \frac{\mathrm{Var}(S)}{(E[S])^2}$ 为服务时间的变异系数平方。

9.3.5 排队网络

Jackson开环网络：

多个排队节点串联或并联，外部到达为Poisson过程。

流量方程：

$$r_j = r_{0j} + \sum_{i=1}^{k} r_i p_{ij} \tag{9.21}$$

每个节点可独立按M/M/1或M/M/c分析。

9.4 随机过程仿真

9.4.1 离散事件仿真

Algorithm 9.3 离散事件仿真框架

输入: 仿真时长 T 或事件数 N

输出: 系统性能统计

初始化:

仿真时钟 $t \leftarrow 0$
事件列表 $E \leftarrow \emptyset$
统计变量初始化
生成初始事件并加入 E
while $t < T$ 且 E 非空 **do**
    从 E 中提取最早事件 (t_e, type)
    $t \leftarrow t_e$
    根据事件类型执行相应逻辑
    更新系统状态
    生成新事件并加入 E
    更新统计数据
end while
计算并输出性能指标

9.4.2 随机数生成

均匀随机数：

- 线性同余法：$X_{n+1} = (aX_n + c) \mod m$
- Mersenne Twister：现代标准算法

非均匀随机数：

- **逆变换法：** $X = F^{-1}(U)$，$U \sim U(0,1)$
- **拒绝采样：** 从提议分布采样后接受/拒绝

- **Box-Muller变换：** 生成正态分布随机数

9.5 风险评估应用

9.5.1 VaR (Value at Risk) 计算

$$\text{VaR}_\alpha = -\inf\{x : P(L \leq x) \geq \alpha\} \tag{9.22}$$

蒙特卡洛方法：

1. 模拟资产收益率情景
2. 计算每种情景下的组合损失
3. 取损失分布的 α 分位数

9.5.2 可靠性分析

系统可靠度：

$$R(t) = P(T > t) = 1 - F(t) \tag{9.23}$$

蒙特卡洛仿真：

- 模拟组件寿命
- 根据系统结构（串联/并联）判断系统失效时间
- 统计可靠度估计

9.6 实际应用建议

1. **样本量选择：**
   - 初步实验：$N = 1000$
   - 正式运行：$N \geq 10000$
   - 高精度需求：$N \geq 100000$
2. **收敛诊断：**
   - 绘制估计值随样本量的变化
   - 多次独立运行比较结果

- 计算置信区间

3. **随机种子管理：**

- 记录种子以便复现
- 不同实验使用不同种子
- 避免使用系统时间的低质量随机数

4. **验证与确认：**

- 与解析解对比（如有）
- 极端情况测试
- 敏感性分析

Part X

微分方程数値解法

Part XI

数据降维算法

Part XII

其他拓展建模算法

Part XIII

复杂系统演化类

Part XIV

博弈论建模算法

Part XV

传染病与种群动力学

Part XVI

离散事件系统仿真

Chapter 10

离散事件系统仿真

10.1 事件步长法

以事件发生时刻驱动仿真时钟推进。

10.2 固定步长法

等时间间隔更新系统状态。

10.3 Petri网

图形化建模工具，包含库所（Place）、变迁（Transition）和令牌（Token）。
基本元素：

- **库所**：表示条件或状态，用圆圈表示
- **变迁**：表示事件或动作，用矩形表示
- **弧**：连接库所和变迁的有向边
- **令牌**：库所中的标记，表示资源或条件满足

触发规则：当变迁的所有输入库所都有足够令牌时，变迁可以触发，消耗输入令牌并产生输出令牌。

Chapter 11

可靠性与故障建模

11.1 故障树分析 (FTA)

自上而下的逻辑分析，从顶事件（系统失效）向下分解为基本事件。

逻辑门：

- **与门 (AND)**：所有输入事件发生时输出事件发生
- **或门 (OR)**：任一输入事件发生时输出事件发生

概率计算：

$$P(A \text{ AND } B) = P(A) \cdot P(B) \quad \text{(独立)} \tag{11.1}$$

$$P(A \text{ OR } B) = P(A) + P(B) - P(A)P(B) \tag{11.2}$$

11.2 威布尔分布

可靠性分析中常用的寿命分布：

$$F(t) = 1 - e^{-(t/\eta)^{\beta}}, \quad f(t) = \frac{\beta}{\eta}\left(\frac{t}{\eta}\right)^{\beta-1} e^{-(t/\eta)^{\beta}} \tag{11.3}$$

其中 β 为形状参数，η 为尺度参数。

- $\beta < 1$：早期失效期
- $\beta = 1$：随机失效期（指数分布）
- $\beta > 1$：磨损失效期

11.3 蒙特卡洛可靠性仿真

通过随机抽样模拟组件失效，统计系统可靠度。

Chapter 12

库存与供应链专用建模

12.1 EOQ经济订货批量

$$Q^* = \sqrt{\frac{2DS}{H}} \tag{12.1}$$

其中 D 为年需求，S 为订货成本，H 为单位持有成本。

总成本：$TC = \frac{D}{Q}S + \frac{Q}{2}H$

12.2 报童模型

单周期随机需求下的最优订货量：

$$Q^* = F^{-1}\left(\frac{c_u}{c_u + c_o}\right) \tag{12.2}$$

其中 c_u 为欠储成本，c_o 为过储成本，F 为需求分布函数。

12.3 (s,S)策略

当库存降至 reorder point s 时，订货至最大库存水平 S。

Chapter 13

数理统计检验与抽样

13.1 假设检验

t检验：比较均值 **F检验**：比较方差 **卡方检验**：拟合优度、独立性检验

13.2 Bootstrap自助法

通过有放回重抽样估计统计量的分布和置信区间。

Algorithm 13.1 Bootstrap置信区间估计

输入: 原始样本 $\{x_1, \ldots, x_n\}$，Bootstrap次数 B

输出: 统计量 θ 的置信区间

1: 计算原始样本统计量 $\hat{\theta}$
2: **for** $b = 1$ to B **do**
3: 有放回抽取 n 个样本，得到 Bootstrap 样本
4: 计算 Bootstrap 统计量 $\hat{\theta}_b^*$
5: **end for**
6: 对 $\{\hat{\theta}_1^*, \ldots, \hat{\theta}_B^*\}$ 排序
7: 取 $\alpha/2$ 和 $1-\alpha/2$ 分位数作为置信区间边界
8: **return** 置信区间

Chapter 14

流体、交通专用建模

14.1 LWR交通流模型

宏观连续模型：

$$\frac{\partial \rho}{\partial t} + \frac{\partial (\rho v)}{\partial x} = 0 \tag{14.1}$$

其中 ρ 为密度，v 为速度。

14.2 IDM智能驾驶员模型

微观跟驰模型：

$$\frac{dv}{dt} = a\left[1 - \left(\frac{v}{v_0}\right)^{\delta} - \left(\frac{s^*}{s}\right)^2\right] \tag{14.2}$$

其中 s 为车间距，s^* 为期望最小间距。

Part XVII

补充智能优化算法

Chapter 15

补充智能优化算法

15.1 人工蜂群算法 (ABC)

模拟蜜蜂采蜜行为，分为三种角色：

- **雇佣蜂**：探索已知蜜源，进行局部搜索
- **观察蜂**：根据雇佣蜂的信息选择蜜源
- **侦查蜂**：随机搜索新蜜源

新解生成：$v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj})$，其中 $\phi_{ij} \in [-1, 1]$。

15.2 布谷鸟搜索 (CS)

基于巢寄生和莱维飞行：

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \alpha \oplus L(\lambda) \tag{15.1}$$

其中 $L(\lambda)$ 为莱维分布，产生长尾随机步长。

15.3 教与学优化 (TLBO)

模拟课堂教学过程：

- **教师阶段**：学生向最优解（教师）学习
- **学习者阶段**：学生之间相互学习

无需调整参数，实现简单。

Chapter 16

模糊拓展与粗糙集衍生

16.1 模糊层次分析法 (FAHP)

用三角模糊数 (l, m, u) 替代AHP中的精确判断值。

计算步骤：

1. 构建模糊判断矩阵
2. 计算模糊权重
3. 去模糊化得到 crisp 权重
4. 一致性检验

16.2 变精度粗糙集 (VPRS)

允许一定程度的分类错误，提高模型的鲁棒性。

Chapter 17

信号与数据处理

17.1 快速傅里叶变换 (FFT)

将时域信号转换为频域：

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i 2\pi k n / N} \tag{17.1}$$

Cooley-Tukey算法将复杂度从 $O(N^2)$ 降至 $O(N \log N)$。

17.2 小波阈值降噪

Algorithm 17.1 小波降噪流程

输入: 含噪信号 x，小波基，分解层数 J

输出: 降噪后信号 $\hat{x}$

1: 对 x 进行 J 层小波分解

2: 估计噪声标准差 σ

3: 计算阈值 $\lambda = \sigma\sqrt{2 \ln N}$

4: 对小波系数应用软阈值或硬阈值

5: 逆小波变换重构信号

6: **return** $\hat{x}$

Chapter 18

组合优化与组合数学

18.1 容斥原理

$$|A_1 \cup \cdots \cup A_n| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \cdots + (-1)^{n+1} |A_1 \cap \cdots \cap A_n| \tag{18.1}$$

18.2 回溯算法

系统性地枚举所有可能解，剪枝不可行分支。

Chapter 19

图像处理类建模

19.1 Otsu大津法

自动确定最佳阈值，最大化类间方差：

$$\sigma_B^2(t) = w_1(t)w_2(t)[\mu_1(t) - \mu_2(t)]^2 \tag{19.1}$$

19.2 形态学运算

- **腐蚀**：$A \ominus B = \{z|B_z \subseteq A\}$
- **膨胀**：$A \oplus B = \{z|(\hat{B})_z \cap A \neq \emptyset\}$
- **开运算**：先腐蚀后膨胀，去除小物体
- **闭运算**：先膨胀后腐蚀，填充小孔

Chapter 20

偏微分方程补充数值解法

20.1 有限体积法 (FVM)

基于守恒律，在控制体积上积分：

$$\frac{d}{dt}\int_V u dV + \oint_S \mathbf{F}\cdot\mathbf{n} dS = 0 \tag{20.1}$$

广泛应用于计算流体力学(CFD)。

20.2 谱方法

用全局光滑基函数（如傅里叶级数、切比雪夫多项式）逼近解，具有指数收敛速度。

Chapter 21

计量经济学建模

21.1 ADF单位根检验

检验时间序列是否平稳：

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^{p} \delta_i \Delta y_{t-i} + \varepsilon_t \tag{21.1}$$

检验 $H_0 : \gamma = 0$（存在单位根，非平稳）。

21.2 协整检验

Engle-Granger两步法：

1. OLS回归：$y_t = \beta x_t + \varepsilon_t$
2. 对残差 $\hat{\varepsilon}_t$ 进行ADF检验

若残差平稳，则变量协整，存在长期均衡关系。

21.3 GARCH模型

波动率建模：

$$r_t = \sigma_t \varepsilon_t, \quad \varepsilon_t \sim N(0,1) \tag{21.2}$$

$$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2 \tag{21.3}$$

捕捉金融收益率的波动聚集效应。

Chapter 22

最优控制与变分类

22.1 变分法

求泛函极值：

$$J[y] = \int_a^b F(x, y, y')dx \tag{22.1}$$

Euler-Lagrange方程：

$$\frac{\partial F}{\partial y} - \frac{d}{dx}\left(\frac{\partial F}{\partial y'}\right) = 0 \tag{22.2}$$

22.2 庞特里亚金极小值原理

对于最优控制问题 $\min \int L(x, u, t)dt$，s.t. $\dot{x} = f(x, u, t)$：
构造Hamiltonian：$H = L + \lambda^T f$
必要条件：

- $\dot{x} = \frac{\partial H}{\partial \lambda}$
- $\dot{\lambda} = -\frac{\partial H}{\partial x}$
- $u^* = \arg\min_u H$

22.3 线性二次调节器 (LQR)

对于线性系统 $\dot{x} = Ax + Bu$，最小化二次代价：

$$J = \int_0^\infty (x^T Q x + u^T R u)dt \tag{22.3}$$

最优控制：$u^* = -Kx$，其中 $K = R^{-1}B^T P$，P 为Riccati方程的解。

Chapter 23

鲁棒优化与不确定规划

23.1 鲁棒线性规划

$$\begin{aligned} \min_{\mathbf{x}} \quad & \max_{\mathbf{c}\in\mathcal{U}_c} \mathbf{c}^T\mathbf{x} \\ \text{s.t.} \quad & A\mathbf{x} \le \mathbf{b}, \quad \forall A \in \mathcal{U}_A \end{aligned} \tag{23.1}$$

保证在所有不确定性实现下约束可行。

23.2 机会约束规划

$$P(g(\mathbf{x}, \boldsymbol{\xi}) \le 0) \ge 1 - \alpha \tag{23.2}$$

约束以概率 $1-\alpha$ 成立，α 为风险水平。

Chapter 24

高级多目标优化算法

24.1 NSGA-II详解

快速非支配排序：

- 对每个个体计算被支配次数 n_p 和支配集合 S_p
- 找出 $n_p = 0$ 的个体作为第一前沿
- 递归找出后续前沿

拥挤距离：

$$d_i^j = \frac{f_j^{i+1} - f_j^{i-1}}{f_j^{max} - f_j^{min}} \tag{24.1}$$

24.2 MOPSO多目标粒子群

维护外部档案存储Pareto最优解，引导粒子搜索方向。

Part XVIII

空间地理与GIS建模

Chapter 25

空间地理与GIS建模

25.1 克里金插值 (Kriging)

地统计最优线性无偏估计(BLUE)：

$$\hat{Z}(\mathbf{s}_0) = \sum_{i=1}^{n} \lambda_i Z(\mathbf{s}_i) \tag{25.1}$$

权重 λ_i 通过变异函数模型确定，最小化估计方差。

变异函数：

$$\gamma(h) = \frac{1}{2} E[(Z(\mathbf{s}) - Z(\mathbf{s}+h))^2] \tag{25.2}$$

常用模型：球状、指数、高斯模型。

25.2 反距离加权 (IDW)

$$\hat{Z}(\mathbf{s}_0) = \frac{\sum_{i=1}^{n} w_i Z(\mathbf{s}_i)}{\sum_{i=1}^{n} w_i}, \quad w_i = \frac{1}{d_i^p} \tag{25.3}$$

其中 d_i 为到已知点的距离，p 为幂参数。

25.3 泰森多边形 (Voronoi Diagram)

将平面划分为若干区域，每个区域内的点到对应生成点的距离最近。

应用：服务半径划分、资源分区、邻近分析。

25.4 空间自相关

全局Moran's I：

$$I = \frac{n}{\sum_i \sum_j w_{ij}} \cdot \frac{\sum_i \sum_j w_{ij}(x_i - \bar{x})(x_j - \bar{x})}{\sum_i (x_i - \bar{x})^2} \tag{25.4}$$

- $I > 0$：空间正相关（聚集）

- $I < 0$：空间负相关（分散）
- $I \approx 0$：空间随机

25.5 CA-Markov耦合模型

结合马尔可链的状态转移概率和元胞自动机的空间规则，预测土地利用变化。

Chapter 26

高级排队与服务系统

26.1 M/M/1排队模型

- 到达过程：Poisson过程，速率 λ
- 服务时间：指数分布，速率 μ
- 单服务台

关键指标：

$$\rho = \lambda/\mu \quad \text{(利用率)} \tag{26.1}$$

$$L = \frac{\rho}{1-\rho} \quad \text{(平均顾客数)} \tag{26.2}$$

$$W = \frac{1}{\mu - \lambda} \quad \text{(平均等待时间)} \tag{26.3}$$

稳定性条件：$\rho < 1$。

26.2 M/M/s模型

s 个并行服务台：

$$P_0 = \left[\sum_{n=0}^{s-1} \frac{(\lambda/\mu)^n}{n!} + \frac{(\lambda/\mu)^s}{s!(1-\rho)} \right]^{-1} \tag{26.4}$$

26.3 Jackson排队网络

开环网络，各节点为M/M/1队列，外部到达为Poisson过程。
平衡方程：

$$r_j = r_{0j} + \sum_{i=1}^{k} r_i p_{ij} \tag{26.5}$$

其中 r_j 为节点 j 的总到达率，p_{ij} 为从 i 到 j 的转移概率。

Chapter 27

生物医学建模专项

27.1 药代动力学房室模型

一室模型：

$$\frac{dC}{dt} = -k_e C, \quad C(t) = C_0 e^{-k_e t} \tag{27.1}$$

二室模型：

$$\frac{dC_1}{dt} = -(k_{10} + k_{12})C_1 + k_{21} C_2 \tag{27.2}$$

$$\frac{dC_2}{dt} = k_{12} C_1 - k_{21} C_2 \tag{27.3}$$

27.2 生存分析

Kaplan-Meier估计：

$$\hat{S}(t) = \prod_{t_i \le t} \left(1 - \frac{d_i}{n_i}\right) \tag{27.4}$$

其中 d_i 为时刻 t_i 的事件数，n_i 为风险集中的个体数。

Cox比例风险模型：

$$h(t|\mathbf{x}) = h_0(t) \exp(\boldsymbol{\beta}^T \mathbf{x}) \tag{27.5}$$

Chapter 28

稳健统计与异常值处理

28.1 M估计

最小化目标函数：

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} \rho(y_i - \mathbf{x}_i^T \boldsymbol{\beta}) \tag{28.1}$$

常用 ρ 函数：Huber、Tukey bisquare等。

28.2 Grubbs检验

检测单个异常值：

$$G = \frac{\max_i |y_i - \bar{y}|}{s} \tag{28.2}$$

若 $G > G_{critical}$，则判定为异常值。

Chapter 29

滤波算法

29.1 卡尔曼滤波 (KF)

线性高斯系统的最优状态估计。

预测步骤：

$$\hat{\mathbf{x}}_{k|k-1} = F_k \hat{\mathbf{x}}_{k-1|k-1} + B_k \mathbf{u}_k \tag{29.1}$$

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q_k \tag{29.2}$$

更新步骤：

$$\mathbf{K}_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R_k)^{-1} \tag{29.3}$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - H_k \hat{\mathbf{x}}_{k|k-1}) \tag{29.4}$$

$$P_{k|k} = (I - \mathbf{K}_k H_k) P_{k|k-1} \tag{29.5}$$

29.2 扩展卡尔曼滤波 (EKF)

对非线性系统进行线性化：

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{w}_{k-1} \tag{29.6}$$

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k \tag{29.7}$$

用Jacobian矩阵 $F_k = \frac{\partial f}{\partial \mathbf{x}}$ 和 $H_k = \frac{\partial h}{\partial \mathbf{x}}$ 近似。

29.3 粒子滤波 (PF)

基于蒙特卡洛采样的非线性非高斯滤波：

1. 采样：从提议分布抽取粒子
2. 加权：根据似然计算权重
3. 重采样：避免粒子退化

Chapter 30

组合博弈与合作博弈

30.1 Shapley值

合作博弈中公平分配联盟收益的方法：

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n-|S|-1)!}{n!}[v(S \cup \{i\}) - v(S)] \tag{30.1}$$

性质：效率、对称性、零玩家、可加性。

Chapter 31

水文与环境建模

31.1 马斯京根河道演算

$$O_2 = C_0 I_2 + C_1 I_1 + C_2 O_1 \tag{31.1}$$

系数 C_0, C_1, C_2 由参数 K（传播时间）和 x（权重因子）确定。

31.2 SCS径流曲线法

$$Q = \frac{(P - I_a)^2}{P - I_a + S}, \quad S = \frac{25400}{CN} - 254 \tag{31.2}$$

其中 P 为降雨量，CN 为曲线数（取决于土壤类型和土地利用）。

Chapter 32

稀疏优化与压缩感知

32.1 Lasso (L1正则化)

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left\{ \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_1 \right\} \tag{32.1}$$

产生稀疏解，用于特征选择。

32.2 Ridge回归 (L2正则化)

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left\{ \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_2^2 \right\} \tag{32.2}$$

解决多重共线性问题。

解析解：$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X} + \lambda I)^{-1}\mathbf{X}^T\mathbf{y}$

32.3 弹性网 (Elastic Net)

结合L1和L2正则化：

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left\{ \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda [\alpha \|\boldsymbol{\beta}\|_1 + \frac{1-\alpha}{2} \|\boldsymbol{\beta}\|_2^2] \right\} \tag{32.3}$$

兼具变量选择和稳定性的优点。

Chapter 33

马尔可夫衍生拓展模型

33.1 隐马尔可夫模型 (HMM)

五元组： (S, O, A, B, π)

- S：隐藏状态集合
- O：观测集合
- A：状态转移矩阵
- B：发射概率矩阵
- π：初始状态分布

三个基本问题：

1. 评估：计算观测序列概率（前向算法）
2. 解码：寻找最可能的状态序列（Viterbi算法）
3. 学习：估计模型参数（Baum-Welch算法）

33.2 吸收马尔可夫链

含有吸收态（一旦进入无法离开）的马尔可夫链。
应用：破产概率、设备报废、疾病进展等。

Chapter 34

小众元启发式优化补充

34.1 水循环算法 (WCA)

模拟自然界水循环过程：

- **雨滴**：初始解
- **溪流**：向河流移动
- **河流**：向海洋（最优解）移动
- **蒸发**：避免局部最优

34.2 热交换优化 (TEO)

基于热力学第二定律，模拟物体间的热交换过程达到热平衡（最优解）。

34.3 原子搜索优化 (ASO)

模拟原子间的相互作用力（吸引力和排斥力）进行优化搜索。

附录 A

数学基础

A.1 线性代数

矩阵运算、特征值、奇异值分解等。

A.2 概率论

随机变量、分布、期望、方差等。

A.3 微积分

导数、积分、多元微积分等。

A.4 优化理论

凸优化、对偶理论、KKT条件等。

附录 B

常用软件工具

B.1 MATLAB

优化工具箱、统计工具箱等。

B.2 Python

NumPy、SciPy、scikit-learn等库。

B.3 R语言

统计建模与可视化。

B.4 LINGO/GUROBI

专业优化求解器。

附录 C

数学建模案例分析

本章通过典型案例分析，展示如何将算法应用于实际问题。

C.1 案例一：城市交通拥堵评估

C.1.1 问题描述

某城市需要评估主要道路的拥堵程度，并提出改善建议。

C.1.2 建模思路

1. 评价指标体系构建：

- 交通流量指标：饱和度、平均车速
- 时间指标：平均延误时间、排队长度
- 空间指标：拥堵里程比例
- 环境指标：尾气排放量

2. 数据收集：

- 线圈检测器数据（流量、速度）
- GPS浮动车数据
- 视频监控数据

3. 评价方法：

- 熵权法确定客观权重
- AHP获取专家主观权重
- 组合权重：$w = \alpha w_{;} + (1-\alpha)w$
- TOPSIS进行综合排序

C.1.3 模型实现

假设5条道路，4个指标的数据：

道路	饱和度	平均车速(km/h)	延误(s)	拥堵里程(%)
R1	0.85	25	45	0.60
R2	0.72	35	30	0.45
R3	0.90	20	55	0.75
R4	0.65	40	20	0.30
R5	0.78	30	35	0.50

表 C.1: 道路交通数据

经过TOPSIS计算，得到拥堵程度排序：R3 ¿ R1 ¿ R5 ¿ R2 ¿ R4

C.1.4 改善建议

- R3道路：增加车道、优化信号灯配时
- R1道路：设置潮汐车道
- 整体：发展公共交通、错峰出行

C.2 案例二：传染病预测与控制

C.2.1 问题背景

某地区爆发新型传染病，需要预测疫情发展趋势并评估防控措施效果。

C.2.2 SEIR模型建立

$$\frac{dS}{dt} = -\beta(1-u_1)SI \tag{C.1}$$

$$\frac{dE}{dt} = \beta(1-u_1)SI - \sigma E \tag{C.2}$$

$$\frac{dI}{dt} = \sigma E - \gamma I - \mu I \tag{C.3}$$

$$\frac{dR}{dt} = \gamma I \tag{C.4}$$

其中 u_1 为社交距离措施强度（0-1）。

C.2.3 参数估计

从早期数据拟合：

- 潜伏期：$1/\sigma = 5$ 天
- 感染期：$1/\gamma = 10$ 天
- 基本再生数：$R_0 = \beta/\gamma = 2.5$

C.2.4 情景分析

情景	峰值感染人数	达峰时间	总感染人数
无干预	50,000	第60天	500,000
轻度干预($u_1 = 0.3$)	30,000	第90天	350,000
中度干预($u_1 = 0.5$)	15,000	第120天	200,000
重度干预($u_1 = 0.7$)	5,000	第180天	80,000

表 C.2: 不同干预措施效果对比

C.2.5 疫苗接种策略

扩展模型加入疫苗接种：

$$\frac{dS}{dt} = -\beta SI - u_v S \tag{C.5}$$

群体免疫阈值：$p_c = 1 - 1/R_0 = 60\%$

C.3 案例三：供应链优化

C.3.1 问题描述

某制造企业需要优化多级供应链网络，最小化总成本。

C.3.2 混合整数规划模型

决策变量：

- x_{ij}：从设施 i 到设施 j 的运输量
- y_i：是否在位置 i 建厂（0-1变量）

- I_t：时期 t 的库存水平

目标函数：

$$\min \sum_i F_i y_i + \sum_{i,j} c_{ij} x_{ij} + \sum_t h_t I_t \tag{C.6}$$

约束条件：

$$\text{产能约束：} \quad \sum_j x_{ij} \leq C_i y_i \tag{C.7}$$

$$\text{需求满足：} \quad \sum_i x_{ij} \geq D_j \tag{C.8}$$

$$\text{库存平衡：} \quad I_t = I_{t-1} + P_t - S_t \tag{C.9}$$

$$\text{非负整数：} \quad x_{ij} \geq 0, y_i \in \{0, 1\} \tag{C.10}$$

C.3.3 求解结果

使用Gurobi求解器：

- 最优建厂方案：选择3个位置
- 年总成本：降低18%
- 投资回收期：2.5年

C.4 案例四：股票价格预测

C.4.1 数据预处理

- 清洗：处理缺失值、异常值
- 特征工程：技术指标（MA、RSI、MACD）
- 标准化：Z-score归一化

C.4.2 LSTM模型构建

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.LSTM(64, return_sequences=True,
                         input_shape=(60, 5)),
    tf.keras.layers.Dropout(0.2),
```

```
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=50, batch_size=32)
```

Listing C.1: LSTM模型代码示例

C.4.3 评估结果

模型	RMSE	MAE	MAPE(%)
ARIMA	3.52	2.81	2.1
BP神经网络	2.85	2.23	1.7
LSTM	2.15	1.68	1.3

表 C.3: 预测模型性能对比

附录 D

算法实现技巧与常见陷阱

D.1 数值计算注意事项

D.1.1 精度问题

- 避免大数减小数导致的有效数字丢失
- 使用双精度浮点数（double）
- 矩阵求逆时使用稳定的数值方法（如LU分解）

D.1.2 收敛性问题

- 迭代算法设置最大迭代次数和收敛容差
- 监测目标函数变化，检测停滞
- 尝试不同的初始值，避免局部最优

D.2 智能优化算法调参指南

D.2.1 遗传算法

参数	推荐范围	调整策略
种群大小	50-200	复杂问题取大值
交叉概率	0.6-0.9	保持多样性
变异概率	0.01-0.1	早熟时增大
精英保留	1-5%	保证收敛

D.2.2 粒子群优化

- 惯性权重：从0.9线性递减到0.4
- 学习因子：$c_1 = c_2 = 2.0$ 或 $c_1 = 2.5, c_2 = 0.5$

- 速度限制：$v_{max} = 0.2\times$ 搜索范围
- 种群大小：20-50

D.2.3 模拟退火

- 初始温度：使初始接受率约80%
- 降温系数：0.85-0.99
- 每温步迭代：10-100次
- 终止温度：10^{-6} 或连续无改进

D.3 机器学习建模最佳实践

D.3.1 数据划分

- 训练集：60-80%
- 验证集：10-20%（调参用）
- 测试集：10-20%（最终评估）
- 时序数据：按时间顺序划分，不随机打乱

D.3.2 过拟合预防

- 正则化：L1/L2
- Dropout：0.2-0.5
- Early stopping：patience=10
- 数据增强：增加训练样本
- 交叉验证：K=5或10

D.3.3 特征工程

- 缺失值处理：删除、填充、插值
- 异常值检测：3σ原则、IQR方法
- 特征缩放：标准化、归一化

- 特征选择：相关性分析、递归消除
- 特征构造：多项式、交互项

D.4 常见错误与避免方法

D.4.1 建模常见错误

1. **忽视数据质量**
   - 错误：直接使用原始数据
   - 正确：先进行数据清洗和探索性分析
2. **模型过于复杂**
   - 错误：盲目使用深度学习
   - 正确：从简单模型开始，逐步增加复杂度
3. **忽视基线模型**
   - 错误：直接比较不同复杂模型
   - 正确：先建立简单基线（如均值预测）
4. **数据泄露**
   - 错误：用未来数据预测过去
   - 正确：严格区分训练和测试时间窗口
5. **过度拟合测试集**
   - 错误：反复调整直到测试集表现最好
   - 正确：测试集只用一次做最终评估

D.5 论文写作建议

D.5.1 模型描述要点

- 清晰定义所有符号和变量
- 说明模型假设和适用范围
- 给出完整的数学公式

- 解释参数含义和取值依据
- 提供算法流程图或伪代码

D.5.2 结果展示技巧

- 使用表格对比不同方案
- 绘制趋势图、柱状图、热力图
- 标注关键数值和结论
- 进行敏感性分析
- 讨论模型局限性

D.5.3 灵敏度分析

对关键参数进行扰动，观察结果变化：

$$\text{灵敏度} = \frac{\Delta\text{输出}}{\Delta\text{输入}} \tag{D.1}$$

D.6 常用软件工具对比

表 D.1: 建模软件工具对比

工具	优势	适用场景	学习曲线
MATLAB	工具箱丰富	信号处理、控制	中等
Python	开源、库多	通用、机器学习	低
R	统计强大	统计分析	中等
LINGO	优化专业	线性/非线性规划	低
SPSS	界面友好	统计分析	低
Excel	普及率高	简单计算	低

D.7 数学建模竞赛策略

D.7.1 时间分配建议（3天赛制）

- **第一天：**
  - 上午：阅读题目，确定选题

  - 下午：文献调研，数据收集
  - 晚上：建立初步模型

- **第二天：**
  - 上午：模型求解，编程实现
  - 下午：结果分析，模型改进
  - 晚上：开始撰写论文

- **第三天：**
  - 上午：完善论文，制作图表
  - 下午：摘要精修，格式检查
  - 晚上：最终审查，提交

D.7.2 团队协作

- 明确分工：建模、编程、写作
- 定期沟通：每4小时同步进度
- 版本控制：使用Git管理代码和文档
- 互相审查：交叉检查对方的工作

D.7.3 摘要写作要点

摘要是评审最关注的部分，应包含：

1. 问题重述（简洁）
2. 建模思路和方法
3. 主要结果（具体数值）
4. 模型特点和创新点
5. 关键词：3-5个